

Who are you, what may you do, and who did what: auth, roles, shares and the audit trail

The win: you can explain, to an interviewer or to yourself, why a password hash must be slow and memory-hard and why it describes its own parameters; why an unknown email must cost the same script as a wrong password and what a caller learns when it does not; why the browser's session is a row you can delete rather than a signed token, and why SHA-256 is right for that token and wrong for the password; what `HttpOnly`, `Secure` and `SameSite=Strict` each defend against; why the role is read from the very row that proves membership and why a route without a guard fails a test; why the room reads the role three times and refuses a viewer's op at its turn, not on arrival; why revocation must beat every lifetime you hand out and how a 60-second token was still too long; why an audit row is written by the action's own statement, why the table has no foreign keys, and how `REVOKE` plus a trigger make it append-only against the app and the owner alike; where this sits against Spring Security and Django; and you can point at the line of code for every claim.

Verification status, honestly. Every commit in this chapter passed lint, the type checker and the unit tests (`password.test`, the auth routes against an in-memory database, the identity strategies, the route table that fails for a route without a guard, the room's refusal of a viewer's op, the contracts, the audit view's words, the config), and each guard was mutation-checked. The integration suites (`api_auth.int` including the real entry point spawned with each `NODE_ENV`, `worker_service-credential.int`, `api_rbac.int`, `sync_role-change.int` including the token race and the sweep, `api_audit.int`, `db_audit.int` and `app-role.int`) and the four end-to-end specs (`sign-in`, `viewer-rejected`, `share-and-revoke`, `audit-view`) are written and committed and *have not been run*: the host was too loaded to run Docker suites beside other work. They run on build/epics before anything merges to main. Where this page says "measured", it was measured by a unit test that ran; where it says "the test asserts", the test exists and is unrun. Section 8 is the by-hand demo to run first.

How to use this page. Every code block is cut from the repo at commit f21d6d2, so what you read is what is committed. Sections 1, 2 and 4 are the ones a security-minded interview will dig into; section 5 is the one a compliance question turns on; section 6 is the comparison you will be asked to make.

```
./init.sh                                     # migrations 0015 to 0017:
credentials, sessions, shares, the trail
pnpm exec vitest run apps/api/src/password.test.ts apps/api/src/auth.test.ts apps/api/src/app.test.ts
apps/web/src/audit.test.ts # the pure halves
open http://localhost:5173/                  # the sign-in form; no ?user= anywhere
```

Try it, once the stack is up: section 8 walks the sign-up, the demotion, the share and its revoke, and an attempt to edit the trail by hand, then the suites, with the exact commands and what each should show.

The map: what identity, roles and a trail mean here

- 1 · Passwords: a slow hash that describes itself, and a sign-in that costs the same for everyone
- 2 · Sessions: a row, a cookie, and why not a JWT
- 3 · Roles at the query: the row that proves membership carries the role
- 4 · Roles in the room: read before the upgrade, re-read at the op's turn, and revocation that beats the token
- 5 · The audit trail: written by the action, and the database refuses to change it
- 6 · Trade-offs and comparisons: where this design sits
- 7 · Testing identity without a browser, and the database's refusals
- 8 · By hand: sign up, demote, revoke, and try to edit the trail
- 9 · What the type system could not see, epic 8 edition

Your turn: three drills

Recall, then test yourself

The map: what identity, roles and a trail mean here

Epic 1 built the tenancy: every org route sits behind one membership query, a stranger and a missing org are the same 404 (F2), and the caller was whoever an `x-dev-user` header said. Epic 8 makes the caller real and asks the three questions that follow from it. **Who are you** (F23): a password

that is hashed slowly and compared in constant time, a session that is a row in Postgres and a cookie the page's script cannot read, and a dev header that outside development opens nothing. **What may you do** (F24, F25): a role read from the very row that proved membership, checked by every route and by the room for every op, and a share that opens one document to someone outside the org, with a revocation that closes their socket within ten seconds and refuses every way back, including a token signed before the revoke. **Who did what** (F26): an audit row written inside the action's own statement, in a table the app's database role cannot update or delete and a trigger refuses to everyone else.

STEP	WHERE	AND	WHAT HAPPENS
1	apps/api/src/password.ts	packages/db/migrations/0015_sign_in.sql	script with its parameters in the stored string; a dummy hash so an unknown email costs a hash too; an opaque 32-byte token kept only as its SHA-256
2	apps/api/src/identity.ts	apps/api/src/app.ts	the session cookie (HttpOnly, Secure, SameSite=Strict); chooseIdentity : only the literal development keeps the header, behind a session; the worker's peers mint their own tokens with the service credential
3	apps/api/src/app.ts	packages/db/src/index.ts	need(role) on every org and document route, and a test that reads the router and fails for a route without one; the role from the membership row; setMember under an advisory lock; shares as a table of their own and the higher of the two roles
4	apps/sync/src/server.ts	packages/lease/src/access.ts	the role read before the upgrade, at the join, and again on every announcement and every 30 s; mayEdit checked when an op's turn comes; Redis pub/sub as a hint that says who changed, never what to
5	packages/db/migrations/0017_audit_log.sql	packages/db/src/index.ts	audit_log with no foreign keys; every audited action writes its row in a CTE of the same statement or in the same transaction; provisionAppRole revokes UPDATE, DELETE and TRUNCATE on every deploy; a trigger refuses the owner too
6	apps/web/src/audit.ts	apps/web/src/AuditView.tsx	a Record over the whole action enum, so a new action does not compile without its sentence; everything an entry says rendered as text

Four ideas carry the epic, and they are what an interviewer will press on. **Authentication is a fixed-cost function of its input** (section 1): whoever you claim to be, sign-in does one slow hash and answers the same bytes, because anything that varies with whether the account exists (time, status, body) is a list of accounts for whoever asks. **A session is a row, not a claim** (section 2): a credential you can revoke is a row you can delete; a signed stateless token cannot be revoked without a deny list, which is a table anyway, so the browser gets a row and only the 60-second sync token stays a JWT. **Authorization lives where the data is fetched** (sections 3 and 4): the query that proves membership returns the role, the route declares what it needs,

and the room re-reads the role where the socket lives, because a socket outlives every token. **An audit trail is evidence only if its writer cannot rewrite it** (section 5): the row is written by the action's own statement, and the database, not the application's manners, refuses every change.

THIS REPO	JAVA WORLD	PYTHON WORLD
script from <code>node:crypto</code> , the parameters in the stored string	Spring Security's <code>DelegatingPasswordEncoder</code> (<code>{bcrypt}</code> , <code>{argon2}</code> , <code>{script}</code> prefixes: the same self-describing idea)	Django's <code>algorithm\$iterations\$salt\$hash</code> (PBKDF2 by default, argon2 optional): the same idea again
a session row and an <code>HttpOnly</code> cookie; sign-out is a <code>DELETE</code>	<code>HttpSession</code> with Spring Session JDBC; <code>SessionRegistry.expireNow()</code>	<code>django_session</code> : an opaque key, a row, an <code>HttpOnly</code> cookie; <code>Session.objects.filter(...).delete()</code>
need(role) per route plus a test over the router	<code>@PreAuthorize("hasRole('OWNER')")</code> and a <code>RoleHierarchy</code> ; <code>authorizeHttpRequests(...anyRequest().denyAll())</code>	<code>@permission_required</code> , <code>PermissionRequiredMixin</code> ; django-guardian for per-object roles
the role re-read in the room on a Redis hint	Spring's <code>WebSocket</code> security checks the handshake only; a role change reaches an open socket by hand	Channels' <code>AuthMiddlewareStack</code> checks at connect; the same gap
audit rows in the action's statement, grants revoked, a trigger	Hibernate Envers writes <code>_AUD</code> tables in the same transaction, but the app role may still <code>UPDATE</code> them	django-simple-history, django-auditlog: histories, not fenced trails, unless the role is

1 • Passwords: a slow hash that describes itself, and a sign-in that costs the same for everyone

A password is a low-entropy secret: people choose them, and an attacker who steals the table will guess them offline at whatever speed the hash allows. So the hash must be *slow*, and, since a GPU has thousands of cores that are each slow, it must also be *memory-hard*, so that every parallel guess pays in RAM as well as time. SHA-256 is neither: it is designed to be fast. `bcrypt` is slow but uses 4 KiB; `script` and `argon2` are both. `script` was chosen because it ships with Node (`node:crypto`), and `argon2id`, OWASP's first choice, would be the pick when a native dependency is acceptable.

FROM `apps/api/src/password.ts`

```
/**
 * E8.1 (F23): passwords and sign-in tokens, with Node's own crypto and nothing else.
 *
 * script, because it is slow AND memory-hard (a GPU farm pays in RAM as well as time), and it ships with Node.
 * N=2^15, r=8: 32 MiB and some tens of milliseconds per guess. The stored string names its parameters, so they
 * can be raised later and every older hash still verifies.
 */
const COST = { N: 2 ** 15, r: 8, p: 1 };
const KEY_BYTES = 32;
const STORED = /^script\$(\d+)\$(\d+)\$(\d+)\$([A-Za-z0-9_-]+)\$([A-Za-z0-9_-]+)$/;

function derive(password: string, salt: Buffer, cost: { N: number; r: number; p: number }): Promise<Buffer> {
  const options: ScriptOptions = { ...cost, maxmem: 256 * cost.N * cost.r };
  return new Promise((resolve, reject) => {
    // NFKC: the same password typed on two keyboards can arrive as two byte sequences (NIST 800-63B asks for this).
    script(password.normalize("NFKC"), salt, KEY_BYTES, options, (err, key) => {
      if (err) reject(err);
      else resolve(key);
    });
  });
}
```

Read the numbers. `script`'s memory is `128 · N · r` bytes: with `N = 2^15` and `r = 8` that is 32 MiB per guess, and some tens of milliseconds on a laptop core. Node's default `maxmem` is exactly 32 MiB and the derivation would throw at the boundary, so the options pass `256 · N · r`: room for the parameters to rise. The stored string names `N`, `r` and `p`, which is what lets the cost go up next year without a migration: an old hash still verifies with the parameters it names, and the row is rewritten at the person's next sign-in if anyone wants that. Django and Spring's `DelegatingPasswordEncoder` store their hashes the same way, for the same reason. `NFKC` is NIST 800-63B's advice: the same password typed on two keyboards can arrive as two byte sequences (`é` as one code point or as `e` plus a combining accent), and without normalising, one of them would be refused.

FROM `apps/api/src/password.ts`

```
export async function hashPassword(password: string): Promise<string> {
  const salt = randomBytes(16);
  const key = await derive(password, salt, COST);
  return `script${String(COST.N)}${String(COST.r)}${String(COST.p)}${salt.toString("base64url")}${key.toString("base64url")}`;
}
```

```

/** Constant time over the derived key. A stored string that is not ours (or asks for absurd work) is simply "no". */
export async function verifyPassword(password: string, stored: string): Promise<boolean> {
  const match = STORED.exec(stored);
  if (!match) return false;
  const [N, r, p] = [Number(match[1]), Number(match[2]), Number(match[3])];
  // A row edited to ask for 2^40 would be a denial of service on every sign-in; ours never exceed these.
  if (N > 2 ** 20 || r > 16 || p > 4) return false;
  const expected = Buffer.from(match[5] ?? "", "base64url");
  if (expected.length !== KEY_BYTES) return false;
  // Parameters script refuses (N not a power of two, say) reject: that is a no as well.
  const key = await derive(password, Buffer.from(match[4] ?? "", "base64url"), { N, r, p }).catch(() => undefined);
  return key !== undefined && timingSafeEqual(key, expected);
}

```

Three decisions in `verifyPassword`, each a small test. **Constant time over the key.** `timingSafeEqual` compares every byte whatever the first mismatch; a plain `===` on Buffers returns at the first difference, and a network attacker with enough samples can read the position of the mismatch out of the response time. Java's `MessageDigest.isEqual` and Python's `hmac.compare_digest` exist for the same reason. **A ceiling on the parameters.** The stored string is data from a database row, and a row that someone edited to ask for `N = 2^40` would turn every sign-in of that account into a denial of service on the api; the ceiling refuses it as a plain `false`. **A refusal by script is a no, not a crash.** `N` that is not a power of two makes Node throw; the `.catch(() => undefined)` turns that into "does not match", so a tampered row cannot 500 the route either.

FROM `apps/api/src/password.ts`

```

let dummy: Promise<string> | undefined;
/**
 * A real hash of a password nobody knows. Sign-in verifies against it when the email has no account, so "no such
 * user" costs exactly one script, as "wrong password" does: the response time does not tell which it was.
 */
export const dummyHash = (): Promise<string> => (dummy ??= hashPassword(randomBytes(32).toString("hex")));

/** 32 random bytes: unguessable. The browser keeps the token; the database keeps only its SHA-256. */
export function newSessionToken(): { token: string; hash: Buffer } {
  const token = randomBytes(32).toString("base64url");
  return { token, hash: hashToken(token) };
}
export const hashToken = (token: string): Buffer => createHash("sha256").update(token).digest();
/** The exact shape `newSessionToken` makes. Anything else is never looked up. */
export const isSessionToken = (value: string | undefined): value is string => value !== undefined && /^[A-Za-z0-9_-]{43}$/.test(value);

```

Now the sign-in route itself, and the decision the whole section turns on: *no user enumeration*. Two branches look alike to a compiler, "no such email" and "wrong password", and the naive code answers the first one at once, without hashing anything. That answer arrives tens of milliseconds sooner, and a caller who times it learns which emails have accounts: the list a credential-stuffing run starts from. So an unknown email is verified against a real script hash of a password nobody knows, both branches cost exactly one hash, and both answer the same bytes. The dummy is computed once and *warmed at startup*, in `buildApp`: were it lazy, the first unknown email would pay for two hashes, which is a tell of its own. Drill 2 plants the naive version.

FROM `apps/api/src/app.ts`

```

// Paid for now, not by the first sign-in with an unknown email (whose extra hash would be a timing tell).
dummyHash().catch(() => undefined);

```

FROM `apps/api/src/app.ts`

```

// Sign-up must say when an email is taken: without email (a non-goal) there is no way to answer "maybe" and
// still create the account. That is the one place an account's existence shows, and it is rate limited.
app.post("/auth/signup", async (c) => {
  const { email, name, password } = await body(c, SignUpBody);
  const allowed = await attempt(`signup:${email.toLowerCase()}`);
  if (!allowed.ok) return limited(c, allowed.retryAfterSeconds, "too_many_attempts");
  const user = await db.signUp({ email, name, passwordHash: await hashPassword(password) });
  return user === "taken" ? fail(c, 409, "email_taken") : signedIn(c, user, 201);
});
// No such email and the wrong password are one answer, in one time: both cost exactly one script.
app.post("/auth/signin", async (c) => {
  const { email, password } = await body(c, SignInBody);
  const allowed = await attempt(`signin:${email.toLowerCase()}`);
  if (!allowed.ok) return limited(c, allowed.retryAfterSeconds, "too_many_attempts");
  const found = await db.credentialsFor(email);
  const matches = await verifyPassword(password, found?.passwordHash ?? (await dummyHash()));
  return found && matches ? signedIn(c, found.user, 200) : fail(c, 401, "invalid_credentials");
});

```

Read `found?.passwordHash ?? (await dummyHash())` and then `found && matches`: the hash is always computed, and the answer still needs both facts, so the dummy can never sign anyone in even if a password happened to match it (it cannot: it is a hash of 32 random bytes). The one place an

account's existence *does* show is sign-up: without an email flow, which SPEC calls a non-goal, there is no way to answer "maybe, check your inbox" and still create the account, so a taken email is a 409, and that route goes through the same `allowAttempt` seam as sign-in, where E9.6's rate limiter plugs in. `SignInBody` is also deliberately looser than `SignUpBody`: a sign-in whose password is shorter than the sign-up rule allows is still just a wrong password, not a 400 that says "that is not even a valid password".

FROM `packages/contracts/src/index.ts`

```
/**
 * What a person calls themselves, shown to everyone in the document (presence). Stricter than 'Name': format
 * characters (a right-to-left override, zero-width joiners) and stacks of combining marks let one name pass for
 * another. Cosmetic, never privilege (actor ids decide), but a person's name should read as what it is.
 */
const DisplayName = Name.refine((name) => !/\p{Cf}/u.test(name), "must not contain invisible formatting characters").refine((name) =>
!/\p{M}{3}/u.test(name), "must not stack combining marks");
/** 8 to 128 characters (NIST 800-63B: a floor, no composition rules; the cap bounds the work one request can ask of the hash). */
const Password = z.string().min(8, "must be at least 8 characters").max(128, "must be at most 128 characters");
export const SignUpBody = z.strictObject({ email: User.shape.email, name: DisplayName, password: Password });
/** Only the caps: a sign-in that breaks the sign-up rules is still just a wrong password, and says nothing more. */
export const SignInBody = z.strictObject({ email: z.string().max(320), password: z.string().max(128) });
```

FROM `apps/api/src/auth.test.ts`

```
test("an unknown email and a wrong password get the same answer, byte for byte, and neither sets a cookie", async () => {
  const { post } = app();
  await post("/auth/signup", ann);
  const unknown = await post("/auth/signin", { email: "nobody@example.com", password: ann.password });
  const wrong = await post("/auth/signin", { email: ann.email, password: "wrong horse battery" });
  const notAnEmail = await post("/auth/signin", { email: "nobody", password: "x" });
  for (const res of [unknown, wrong, notAnEmail]) {
    expect(res.status).toBe(401);
    expect(res.headers.get("set-cookie")).toBeNull();
  }
  const [a, b, c] = await Promise.all([unknown.text(), wrong.text(), notAnEmail.text()]);
  expect(a).toBe(JSON.stringify({ error: "invalid_credentials" }));
  expect(b).toBe(a);
  expect(c).toBe(a);
  expect([...unknown.headers.entries()]).toEqual([...wrong.headers.entries()]);
});

test("an unknown email costs a password hash too: its answer is not measurably faster than a wrong password's", async () => {
  const { post } = app();
  await post("/auth/signup", ann);
  const time = async (email: string) => {
    const started = performance.now();
    await post("/auth/signin", { email, password: "wrong horse battery" });
    return performance.now() - started;
  };
  await time("warm@example.com");
  const median = (xs: number[]) => xs.sort((x, y) => x - y)[Math.floor(xs.length / 2)] ?? 0;
  const unknown: number[] = [];
  const wrong: number[] = [];
  for (let i = 0; i < 5; i++) {
    unknown.push(await time(`nobody-${String(i)}@example.com`));
    wrong.push(await time(ann.email));
  }
  // Without the dummy hash the unknown email answers in well under a millisecond, the other in tens of them.
  expect(median(unknown)).toBeGreaterThan(median(wrong) * 0.3);
});
```

FROM `packages/db/src/index.ts`

```
signUp: async ({ email, name, passwordHash }) =>
  (await one(
    UserRow,
    // One statement: a user never exists without the password they signed up with, and two sign-ups of one
    // email race on the unique key, where exactly one wins.
    "with u as (insert into users (email, name) values (lower($1), $2) on conflict (email) do nothing returning id, email, name), " +
    "c as (insert into credentials (user_id, password_hash) select id, $3 from u) select * from u",
    [User.shape.email.parse(email), Name.parse(name), passwordHash],
  )) ?? "taken",
```

FROM `packages/db/migrations/0015_sign_in.sql`

```
-- E8.1 (F23): sign up, sign in, sign out. Two tables of their own, never columns on users: a user created
-- before sign-in existed (the development header's) simply has no credential, and cannot sign in as anyone.

-- One password per user, stored only as a script hash that names its own parameters, so they can be raised later
-- without a migration (a hash with the old ones still verifies).
```

```

create table credentials (
  user_id      uuid primary key references users (id) on delete cascade,
  password_hash text not null check (password_hash ~ '^script\[0-9]+\[0-9]+\[0-9]+\[A-Za-z0-9_-]+\[A-Za-z0-9_-]+\$',
  created_at   timestampz not null default now()
);

-- A signed-in browser. The cookie holds 32 random bytes; this row holds only their SHA-256, so a read of the
-- table (a backup, a log of a query) hands nobody a live session. Signing out deletes the row: revoked at once.
create table auth_sessions (
  id          uuid primary key default gen_random_uuid(),
  user_id     uuid not null references users (id) on delete cascade,
  token_hash  bytea not null unique check (length(token_hash) = 32),
  created_at  timestampz not null default now(),
  expires_at  timestampz not null check (expires_at > created_at)
);
create index auth_sessions_user on auth_sessions (user_id);

```

The migration keeps credentials in a table of their own rather than as columns on `users`: a user created before sign-in existed, by the development header, simply has no row here, and sign-in for them verifies against the dummy like everyone else's miss. The `CHECK` on `password_hash` is the stored format written a second time, in SQL: a row that is not a script string of ours cannot be inserted, whatever the application did.

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>script(password, salt, 32, options, callback)</code> wrapped in <code>new Promise</code>	<code>SCryptPasswordEncoder</code> ; or BouncyCastle's <code>SCrypt.generate</code>	<code>hashlib.scrypt(password, salt=..., n=..., x=..., p=..., maxmem=...)</code>
<code>timingSafeEqual(key, expected)</code> on two Buffers of equal length	<code>MessageDigest.isEqual(a, b)</code>	<code>hmac.compare_digest(a, b)</code>
<code>password.normalize("NFKC")</code>	<code>Normalizer.normalize(s, Normalizer.Form.NFKC)</code>	<code>unicodedata.normalize("NFKC", s)</code>
<code>dummy ??= hashPassword(...)</code> : a memoised promise (the assignment runs once, everyone awaits the same one)	a <code>Supplier</code> behind an <code>AtomicReference</code> , or a static holder	<code>functools.cache</code> on a coroutine factory; or a module-level future
<code>(value): value is string =></code> : a type predicate, so <code>token</code> narrows after the check	no equivalent; an <code>Optional</code> and a boolean	<code>TypeGuard[str]</code> from <code>typing</code>
<code>randomBytes(16)</code> for the salt, <code>base64url</code> without padding	<code>SecureRandom.nextBytes</code> , <code>Base64.getUrlEncoder().withoutPadding()</code>	<code>secrets.token_bytes(16)</code> , <code>base64.urlsafe_b64encode(...).rstrip(b"=")</code>

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The dummy hash is warmed at startup, not on the first miss.

The obvious shape is a lazy `dummyHash()` that computes on first use. Then the first sign-in with an unknown email costs two script derivations, the dummy's and the comparison's, and answers noticeably later than every later miss: one request that stands out is enough to say "that email was the first unknown one". `buildApp` starts the computation and drops the promise; `dummy ??=` makes every later caller await the same one.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The parameter ceiling exists because a stored hash is input, not truth.

Nothing in the type system distinguishes a hash the app wrote from one an operator pasted into a row. Without the ceiling, a row asking for `N = 2^40` would make every sign-in of that account allocate terabytes and time out, and a database write becomes a way to take the api down one account at a time. The test feeds `script$1099511627776$...` and asks for a plain `false`, and the same test refuses `N = 3` (not a power of two) without a throw.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Sign-in that validated like sign-up was rejected.

Reusing `SignUpBody` for sign-in would answer 400 "must be at least 8 characters" to a short password, which tells the caller something about the account's rules before any hash is computed, and separates "malformed" from "wrong" in the status code. `SignInBody` keeps only the caps, so that everything that is not a match is one 401 with one body.

The principle: an authentication endpoint is a function whose cost and whose answer must not depend on the secret it is protecting. Slow hashing protects the table if it leaks; constant-time comparison protects the comparison; the dummy hash protects the *existence* of the account. Miss any one and an attacker learns something for free. The rate limiter that E9.6 plugs into `allowAttempt` is the fourth layer, against online guessing at scrypt's price per try.

2 · Sessions: a row, a cookie, and why not a JWT

Signing in produces a session, and the first design question is what a session *is*. This repo already had one signed stateless token (E1.5's HS256 JWT that opens a document's socket for 60 seconds), so the tempting answer was a second one for the browser. It was rejected for one reason that decides most session designs: **revocation**. A signed token is valid until it expires, wherever it is; to revoke one early you need a deny list the server consults on every request, and a deny list is a table. So the browser's session is a table row from the start, and the token is 32 random bytes that mean nothing except "look me up".

FROM `apps/api/src/identity.ts`

```
/** The cookie a signed-in browser carries (E8.1). HttpOnly and SameSite=Strict: see app.ts, where it is set. */
export const SESSION_COOKIE = "noon_session";

/**
 * E8.1 (F23): the caller is whoever holds a live sign-in session. The cookie's token is hashed and looked up;
 * a malformed one is never sent to the database. One indexed READ per request, no write (the E1.4 finding):
 * the expiry is fixed at sign-in, never slid forward.
 */
export const sessionIdentity: Identify = async (c, db) => {
  const token = getCookie(c, SESSION_COOKIE);
  return isSessionToken(token) ? db.userForSession(hashToken(token)) : undefined;
};

/** Development: a session first, so sign-in works there as it does everywhere, and the header when there is none. */
const developmentIdentity: Identify = async (c, db) => (await sessionIdentity(c, db)) ?? devHeaderIdentity(c, db);

/**
 * Only the literal "development" turns the header on. "test" does not: an image built in CI and
 * promoted with NODE_ENV=test must not accept it. Tests pass the strategy they want directly.
 */
export const chooseIdentity = (nodeEnv: "development" | "test" | "production"): Identify =>
  nodeEnv === "development" ? developmentIdentity : sessionIdentity;
```

Two hashes appear in this epic and they are chosen on opposite principles. The *password* gets scrypt, slow and memory-hard, because a password can be guessed. The *session token* gets one SHA-256, fast, because 256 random bits cannot be guessed at any speed: the hash is there so that a read of the table (a backup, a log line with a query in it) hands nobody a live session, not to slow an attacker down. Hashing a high-entropy token with a KDF would cost tens of milliseconds on *every request* for no security. `isSessionToken` refuses anything that is not exactly the 43 base64url characters `newSessionToken` makes, so a malformed cookie never reaches the database.

FROM `apps/api/src/app.ts`

```
// E8.1 (F23): sign up, sign in, sign out. The session is a cookie the page's script cannot read (HttpOnly)
// and the browser never sends from another site (SameSite=Strict): that is the CSRF defence for every
// write, including POST /documents/:id/session, which reads no body and so skips body()'s JSON check.
const cookieOptions = { httpOnly: true, secure: signIn.secureCookie, sameSite: "Strict", path: "/" } as const;
async function signIn(c: Context, user: User, status: 200 | 201) {
  // A new token every time: nothing a browser held before signing in (a planted cookie) becomes a session.
  const { token, hash } = newSessionToken();
  await db.startSession({ userId: user.id, tokenHash: hash, ttlSeconds: signIn.ttlSeconds });
  setCookie(c, SESSION_COOKIE, token, { ...cookieOptions, maxAge: signIn.ttlSeconds });
  return c.json(user, status);
}
```

FROM `apps/api/src/app.ts`

```
// Public, and always 204: signing out twice, or with a cookie that expired, is not an error. The row goes,
// so a copy of the token (another tab, a stolen cookie) stops working on its very next request.
app.post("/auth/signout", async (c) => {
```

```
const token = getCookie(c, SESSION_COOKIE);
if (isSessionToken(token)) await db.endSession(hashToken(token));
deleteCookie(c, SESSION_COOKIE, cookieOptions);
return c.body(null, 204);
});
// Public, and 200 either way: a signed-out home page is not an error (a 401 here would be one in every console).
app.get("/auth/me", async (c) => c.json({ user: (await identify(c, db)) ?? null } satisfies Me));
```

Each cookie flag is one attack. **HttpOnly** : a script on the page cannot read it, so a cross-site-scripting bug cannot exfiltrate the session (it can still act as the user while the page is open, which is why the rest of the app renders every string from the wire as text). **Secure** : the browser never sends it over plain http, except in development, where the app is served on `localhost` over http and a Secure cookie would never come back; `config.ts` turns it off for the literal `development` and nothing else. **SameSite=Strict** : the browser does not attach it to any request that starts from another site, which is the whole CSRF defence here. Epic 1 already refused bodies that are not `application/json`, which stops the form-post variant of CSRF; but `POST /documents/:id/session` reads no body and so never passed through `body()`, and only the cookie attribute protects it. The price of Strict is that a person arriving from a link in an email is signed out for that first navigation; the design accepts it.

`signedIn` mints a *new* token on every sign-in rather than reusing one a browser might already hold: the session-fixation attack is to plant a known cookie in a victim's browser and wait for them to sign in with it. And the expiry is fixed at sign-in, never slid forward on use, for the reason E1.4 found about the dev header: a sliding expiry makes every request a write (an `UPDATE` of `expires_at`), which takes a row lock and rules out a read replica; a fixed one makes `userForSession` a pure indexed read. Seven days is the trade.

FROM `packages/db/src/index.ts`

```
startSession: async ({ userId, tokenHash, ttlSeconds }) => {
  if (!isId(userId)) throw new Error("cannot start a session: invalid user id");
  await pool.query(
    // ponytail: expired rows are swept at the owner's next sign-in; ceiling: someone who never signs in again
    // keeps dead rows; upgrade: a periodic delete by expires_at.
    "with gone as (delete from auth_sessions where user_id = $1 and expires_at <= now()), " +
    "s as (insert into auth_sessions (user_id, token_hash, expires_at) values ($1, $2, now()) + make_interval(secs => $3)) returning user_id) " +
    // One row per org, not one global row: an org's trail shows its members' sign-ins, and nobody else's.
    `insert into audit_log ${AUDIT_COLUMNS} select m.org_id, 'user', u.id, u.email, 'signed_in', null, '{}' from s join users u on u.id = s.user_id join memberships m on m.user_id = u.id`,
    [userId, tokenHash, ttlSeconds],
  );
},

userForSession: async (tokenHash) =>
  one(UserRow, "select u.id, u.email, u.name from auth_sessions s join users u on u.id = s.user_id where s.token_hash = $1 and s.expires_at > now()", [tokenHash]),

endSession: async (tokenHash) => {
  await pool.query("delete from auth_sessions where token_hash = $1", [tokenHash]);
},
```

One statement does three things: sweeps this user's expired rows (a ponytail: someone who never signs in again keeps dead rows, and a periodic delete is the upgrade), inserts the new session, and writes the sign-in into the audit trail of every org the user belongs to, which is section 5's rule applied here: the row cannot exist without the event, nor the event without the row. A writable CTE (`with gone as (delete ...)`, `s as (insert ... returning ...)` `insert ...`) is Postgres's way to make several writes one statement, and therefore one atomic step, without an explicit transaction.

QUESTION	A SESSION ROW (THE BROWSER)	A SIGNED STATELESS TOKEN (A JWT)	NOTES
What revocation costs	one <code>DELETE</code> ; the token is dead on its next request	impossible before <code>exp</code> without a deny list, which is a table with the same shape as the session table	
What each request costs	one indexed read of <code>auth_sessions</code> by <code>token_hash</code>	a signature check, no I/O	the JWT's real advantage; it buys nothing here, where every route reads Postgres anyway
What a leaked database gives	SHA-256 of tokens: nothing usable	nothing, but a leaked <i>signing key</i> gives every session at once, and rotating it signs everyone out	
Where the truth lives	Postgres, which every api process reads	in the token; several processes agree without sharing anything	the property that made E1.5 pick a JWT for the sync token: the api signs, the sync node verifies, and they never call each other
Lifetime	7 days, fixed at sign-in	as short as the deny-list gap you accept; the sync token lives 60 s for exactly this reason	section 4 is about the one place 60 s was still too long

The dev header, kept in its place. Epic 1's `x-dev-user` header made tenancy real before any sign-in existed. E8.1 keeps it for scripts, the chaos runs, the tunnel demo and "two browser windows as two people", but only where `NODE_ENV` is the literal `development`, and only *behind* a session: a signed-in browser is who it signed in as whatever header it also sends. `test` does not count as development, on purpose: an image built in CI and promoted with `NODE_ENV=test` must not accept the header. Unset means production, the safe side.

FROM `apps/api/src/identity.ts`

```
/**
 * Development only (SPEC §2.16): the caller names themselves in a header and is created on first
 * sight. It exists so that tenancy and attribution are real from epic 1. Since E8.1 every other
 * NODE_ENV knows only sign-in sessions; development keeps the header (scripts, the tunnel demo, two
 * browser windows as two people) behind a real session.
 *
 * Known costs, accepted because this never runs outside a developer's machine: anyone who can reach
 * the api can be anyone and can create users, and every request WRITES (the upsert), so it takes a
 * row lock and cannot run against a read replica. `sessionIdentity` below is a pure read.
 */
export const devHeaderIdentity: Identify = async (c, db) => {
  const email = User.shape.email.safeParse(c.req.header("x-dev-user"));
  if (!email.success) return undefined;
  // The display name is the local part, cut to what `Name` allows: a valid address may have a
  // local part far longer than a name, and that must not turn a request into a 500.
  const local = (email.data.split("@")[0] ?? "").slice(0, 200);
  return db.upsertUser({ email: email.data, name: Name.safeParse(local).data ?? "user" });
};
```

FROM `apps/api/src/config.ts`

```
// Unset means production: the safe side. Anything that relaxes security must be asked for by name.
NODE_ENV: z.enum(["development", "test", "production"]).default("production"),
```

FROM `apps/api/src/config.ts`

```
// Long enough to open a socket, short enough that a leaked token is useless almost at once.
const SESSION_TTL_SECONDS = 60;
/** How long a sign-in lasts (E8.1). Fixed at sign-in, never slid: checking a session must stay a pure read. */
export const SIGN_IN_TTL_SECONDS = 7 * 24 * 60 * 60;
```

FROM `apps/api/src/main.ts`

```
const identify = chooseIdentity(config.nodeEnv);
if (config.nodeEnv === "development") {
  // A misconfigured deployment should at least be unmissable in its own logs.
  process.stderr.write("DEV IDENTITY: the x-dev-user header authenticates any caller without a session. Never run this outside
development.\n");
}
```

Programs are not people. The worker's AI and git peers never used the header: they hold `SESSION_TOKEN_SECRET`, the same secret the api signs sync tokens with, and mint their own 60-second token for the actor they are (`agent` plus the run's id; `git` plus the commit). That is a *service credential*: a process authenticates as itself, and the token's `knd` claim, which only a holder of the secret can set, is what lets the room stamp "the AI did this" on an op in a way no browser can forge. The alternative, routing the worker's peers through the api with a new kind of principal, was rejected as an api hop on the worker's hot path for no gain. The integration test `service-credential.int` proves both peers still reach the room behind a session-only api.

FROM `packages/session-token/src/index.ts`

```
// A session token lets a peer open ONE document's WebSocket for a short time. The api signs it,
// the sync server verifies it: they share a secret and never have to call each other.
//
// The format is a standard HS256 JWT (header.payload.signature, base64url), so any JWT tool can
// read one. Two things are deliberately NOT standard-library-of-the-internet behaviour:
// - the algorithm is PINNED. The token's own "alg" header is never used to choose how to verify,
//   which is what makes the "alg: none" and algorithm-swap attacks impossible here;
// - verification returns a reason instead of throwing, because a bad token is an expected input;
// - exactly ONE string verifies per token (see the canonical-encoding check below).
// The header is compared byte for byte, so any other signer of ours must emit exactly these bytes.
// The role is deliberately NOT a claim: a token is checked once, at connect, and the socket then
// lives for hours, so a role in it would be stale. The sync server reads the role itself (E8.2).
```

FROM `apps/worker/src/ai.ts`

```
session: async () => ({
  wsUrl: await roomUrl(sessions, job.documentId),
  token: signSessionToken({ userId, orgId: job.orgId, documentId: job.documentId, secret: sessions.secret, ttlSeconds: 60, actor: {
kind: "agent", runId: job.id } }),
```

```
    }},
  });
```

FROM `apps/api/src/auth.test.ts`

```
test("without a session every route but the probes and the three sign-in routes is 401, and the dev header changes nothing", async () => {
  const { get, post } = app();
  expect(await get("/orgs")).status().toBe(401);
  expect(await get("/orgs", { "x-dev-user": "ann@example.com" })).status().toBe(401);
  expect(await get("/orgs", { cookie: "noon_session=not-a-token" })).status().toBe(401);
  expect(await post("/documents/00000000-0000-4000-8000-000000000000/session")).status().toBe(401);
  expect(ErrorBody.parse(await (await get("/orgs")).json()).error).toBe("unauthenticated");
  // Asking who you are is not an error when you are nobody: a signed-out home page must not log a 401.
  const me = await get("/auth/me", { "x-dev-user": "ann@example.com" });
  expect(me.status).toBe(200);
  expect(await me.json()).toEqual({ user: null });
});
```

FROM `apps/api/src/auth.int.test.ts`

```
// The REAL entry point in a REAL process, as main.int.test.ts does: a test that hands startServer a strategy
// cannot prove main.ts wires the right one.
describe("integration:dev-header-no-longer-authenticates", () => {
  const spawnApi = useApiProcess();
  // This file's own migrated schema (libpq's options in the URL), so the child sees the sign-in tables whatever
  // state the shared database is in, and every row it writes goes when the schema is dropped.
  const boot = async (env: Record<string, string>): Promise<string> =>
    (await spawnApi({ DATABASE_URL: `${TEST_DATABASE_URL}${TEST_DATABASE_URL.includes("?") ? "&" : "?"}options=${encodeURIComponent(`-c
search_path=${t.schema}`)}', ...env })).url;

  test.each([["unset", {}], ["production", { NODE_ENV: "production" }], ["test", { NODE_ENV: "test" }]])(
    "with NODE_ENV %s the header is 401 on every route, and signing in is the way in",
    async (_label, env) => {
      const url = await boot(env);
      const header = { "x-dev-user": "header-user@example.com" };
      expect(await (await call(url, "GET", "/auth/me", { headers: header })).json()).toEqual({ user: null });
      for (const [method, path] of [
        ["GET", "/orgs"], ["POST", "/orgs"], ["POST", "/documents/00000000-0000-4000-8000-000000000000/session"]
      ]) as const {
        const res = await call(url, method, path, { headers: header, ...(method === "POST" && path === "/orgs" ? { body: { name: "x" } } : {}));
        expect(res.status, `${method} ${path}`).toBe(401);
      }
      const email = `main-${String(Date.now())}-${String(Math.random()).slice(2, 8)}@example.com`;
      const up = await call(url, "POST", "/auth/signup", { body: { email, name: "Main", password: "correct horse battery" } });
      expect(up.status).toBe(201);
      expect(up.headers.get("set-cookie")).toContain("Secure");
      // With a session AND someone else's header, the session decides: the header is not even a tiebreak.
      const me = Me.parse(await (await call(url, "GET", "/auth/me", { cookie: cookieOf(up), headers: header })).json());
      expect(me.user?.email).toBe(email);
    },
  );
});
```

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>setCookie(c, name, value, { httpOnly, secure, sameSite: "Strict", path: "/" })</code>	<code>ResponseCookie.from(name, value).httpOnly(true).secure(true).sameSite("Strict")</code>	<code>response.set_cookie(name, value, httponly=True, secure=True, samesite="Strict")</code>
<code>const cookieOptions = { ... } as const</code> : a literal type, so <code>sameSite</code> is <code>"Strict"</code> , not <code>string</code>	a <code>record</code> with final fields	a <code>Final</code> dict, or a frozen dataclass
<code>randomBytes(32).toString("base64url")</code>	<code>SecureRandom + Base64.getUrlEncoder().withoutPadding()</code>	<code>secrets.token_urlsafe(32)</code>
<code>createHash("sha256").update(token).digest()</code> : a <code>Buffer</code> , stored as <code>bytea</code>	<code>MessageDigest.getInstance("SHA-256")</code>	<code>hashlib.sha256(token.encode("utf-8")).digest("hex")</code>
<code>with gone as (delete ...), s as (insert ... returning) insert ...</code> : three writes, one statement	the same SQL through JDBC; JPA cannot express a writable CTE	the same SQL with psycopg; the CTE is expressed with <code>with ... as ...</code>
<code>nodeEnv === "development" ? developmentIdentity : sessionId</code> : the strategy chosen once, at boot	a <code>@Profile("dev")</code> bean	<code>settings.DEBUG</code> - gated middleware danger of <code>DEBUG=True</code> in production

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A stateless signed token for the browser was rejected.

It would have reused E1.5's package and needed no table. But sign-out would then be a client-side deletion only: a copy of the token in another tab, or a stolen one, works until it expires, and the only fix is a server-side list of revoked tokens checked on every request, which is a session table with worse ergonomics. The row won; the JWT stays where its stateless property is the point, between the api and the sync node.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A sliding expiry was rejected: it would have made every request a write.

Extending the session on each use feels friendlier. It turns `userForSession` into an `UPDATE ... RETURNING`, which takes a row lock per request and cannot be served from a read replica, the exact cost E1.4 found in the dev header's upsert. Seven days fixed at sign-in keeps identity a pure read; the trade is that a person is signed out a week after signing in, whatever they did meanwhile.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The header behind a session in development, not switched off, and not on for test.

Two pressures pulled in opposite directions: scripts and chaos runs need to act as several people without a browser, and a promoted image must never accept a header that lets anyone be anyone. The answer is the literal `"development"` and nothing else, with the session checked first even there, and a loud line in the log when the header is on. The integration test spawns the real `main.ts` with `NODE_ENV` unset, `production` and `test`, and asks 401 of every route with the header.

Principal-level point: pick the hash by the entropy of what it protects, and pick the token by how you will revoke it. Low entropy (a password): a KDF, slow and memory-hard. High entropy (32 random bytes): one fast hash, for the table's sake. Must be revocable at once (a browser session): a row. Must work across processes that never talk, for seconds (the sync token): a signature, with a short life as the only revocation, and section 4 shows what to do when even that is too long.

3 · Roles at the query: the row that proves membership carries the role

Epic 1's door is a join: `select o.*, m.role from orgs o join memberships m on m.org_id = o.id where o.id = $1 and m.user_id = $2`. A stranger and a missing org both get no row, and no row is a 404. E8.2 adds one column to what that query already returns, the role, and puts it on the request context next to the org. Nothing about the caller's authorisation is fetched separately: the same read that says "you may see this org" says "as a viewer". Then each route declares the least role it needs.

FROM `packages/contracts/src/index.ts`

```
// --- Roles (F24) -----
/** A member's role in an org. A viewer reads everything and changes nothing; only an owner changes roles. */
export const Role = z.enum(["owner", "editor", "viewer"]);
export type Role = z.infer<typeof Role>;
const RANK: Record<Role, number> = { viewer: 0, editor: 1, owner: 2 };
/** Does 'role' include everything 'need' may do? The roles are nested: owner > editor > viewer. */
export const includes = (role: Role, need: Role): boolean => RANK[role] >= RANK[need];
export const Member = z.object({ userId: Id, email: User.shape.email, name: Name, role: Role });
export type Member = z.infer<typeof Member>;
/**
 * E10.8: one page of GET /orgs/:orgId/members (every member, oldest first) or of GET /documents/:id/shares (the
 * document's shares, owners only: a share names someone outside the org). A share is a Member whose role is never owner.
 */
export const MemberPage = z.strictObject({ items: z.array(Member), nextCursor: z.string().nullable() });
export type MemberPage = z.infer<typeof MemberPage>;
```

FROM `apps/api/src/app.ts`

```
/** Every guard 'need' made, and the role it asks for: how app.test.ts finds a route that declares none. */
export const GUARDS = new WeakMap<object, Role>();
/**
 * E8.2 (F24): the route needs at least 'role' in the org (the parent's middleware put the caller's role on the
 * context, from the same row that proved membership). A member without it gets 403: they may see the thing, so
 * "not found" would be a lie; a stranger never gets this far (404, F2). Every org and document route declares one:
 * app.test.ts fails for a route that does not, so a new route is closed until someone decides who may use it.
 */
const need = (role: Role) => {
```

```
const guard = createMiddleware<{ Variables: { role: Role } }>(async (c, next) => {
  if (!includes(c.var.role, role)) return fail(c, 403, "forbidden");
  await next();
});
GUARDS.set(guard, role);
return guard;
};
```

`need` is a middleware factory typed over the context variable `role`, so `c.var.role` is a `Role`, never `any`, and it records itself in a `WeakMap` keyed by the middleware function. That map is for one test, and the test is the important half of the design: **default deny**. A handler compiles perfectly well without a guard, and in Spring or Django a route that forgot its `@PreAuthorize` or its decorator is open until someone notices. Here `app.test.ts` walks `app.routes`, looks every org and document route up in the map, and compares the whole table to the one it expects. A new route without `need` shows up as `undefined` and the test is red until someone decides who may call it; a role changed without meaning to shows up too.

FROM `apps/api/src/app.test.ts`

```
// E8.2 (F24): default deny. Every route about one org or one document names the least role it needs, and this is
// the table of them: a new route without a 'need' fails here, and so does a role changed without anyone meaning to.
test("every org and document route declares the role it needs, and they are these", () => {
  const app = buildApp({ db: {} as Db, identify: () => Promise.resolve(undefined), sessions: { secret: "unused-in-this-test-0123456789abcdef", sync: { kind: "one", url: "ws://sync.test", ttlSeconds: 60 }, enqueue: () => Promise.resolve() });
  const guarded = new Map<string, string | undefined>();
  for (const { method, path, handler } of app.routes) {
    if (method === "ALL" || !/^\/(orgs\/:orgId|documents\/:id)(\/|$)/.test(path)) continue;
    const key = `${method} ${path}`;
    guarded.set(key, guarded.get(key) ?? GUARDS.get(handler));
  }
  expect(Object.fromEntries(guarded)).toEqual({
    "GET /orgs/:orgId": "viewer",
    "GET /orgs/:orgId/members": "viewer",
    "PUT /orgs/:orgId/members": "owner",
    "POST /orgs/:orgId/workspaces": "editor",
    "GET /orgs/:orgId/workspaces": "viewer",
    "GET /orgs/:orgId/workspaces/:id": "viewer",
    "POST /orgs/:orgId/workspaces/:id/documents": "editor",
    "GET /orgs/:orgId/workspaces/:id/documents": "viewer",
    "GET /orgs/:orgId/documents/:id": "viewer",
    "GET /orgs/:orgId/usage": "owner",
    "GET /orgs/:orgId/audit": "owner",
    "POST /documents/:id/session": "viewer",
    "GET /documents/:id/shares": "owner",
    "PUT /documents/:id/shares": "owner",
    "DELETE /documents/:id/shares/:userId": "owner",
    "POST /documents/:id/runs": "editor",
    "POST /documents/:id/runs/:runId/cancel": "editor",
    "GET /documents/:id/runs/:runId": "viewer",
    "GET /documents/:id/run": "viewer",
    "POST /documents/:id/preview": "viewer",
    "GET /documents/:id/preview": "viewer",
    "GET /documents/:id/conflict": "viewer",
    "POST /documents/:id/ship": "editor",
    "GET /documents/:id/ship": "viewer",
  });
});
```

FROM `apps/api/src/app.ts`

```
// EVERYTHING about one org lives behind this middleware, including reading the org itself: it is
// the one place that decides whether the caller may see this org. Not a member and no such org
// are the same answer, 404, so a response never confirms that someone else's org exists (F2).
// The same row carries the caller's role, which each route's 'need' checks (E8.2).
const org = new Hono<{ Variables: { user: User; org: Org; role: Role; scope: ReturnType<Db["forOrg"]> } }>();
org.use(async (c, next) => {
  const found = await db.getOrgForMember(c.req.param("orgId") ?? "", c.var.user.id);
  if (!found) return notFound(c);
  c.set("org", found.org);
  c.set("role", found.role);
  c.set("scope", db.forOrg(found.org.id));
  await next();
});

org.get("/", need("viewer"), (c) => c.json(c.var.org));
```

Two status codes, and the rule for choosing between them. A *member* without the role gets **403**: they can already see the thing exists (they are in the org), so "not found" would be a lie that helps nobody. A *stranger* still gets **404** from the middleware before any `need` runs, because the first rule of tenancy (F2) is that a response never confirms another tenant's data exists. The rbac matrix test drives every route as an owner, an editor, a viewer and a stranger, least privileged first so that a refused write cannot leave anything the next row would see.

FROM apps/api/src/rbac.int.test.ts

```
test("every route, every role: reads for every member, writes for editors, roles and usage for owners, 404 for a stranger", async () => {
  const { org, ws, doc, run } = await world();
  const strangerId = Me.parse((await call("stranger", "GET", "/auth/me")).json).user?.id ?? "";
  // [method, path, body, least role]. Each write is one a viewer could do harm with; the stranger row is implied.
  const routes: [string, string, unknown, Role][] = [
    ["GET", `'/orgs/${org.id}'`, undefined, "viewer"],
    ["GET", `'/orgs/${org.id}/members'`, undefined, "viewer"], // E10.8
    ["GET", `'/orgs/${org.id}/workspaces'`, undefined, "viewer"],
    ["GET", `'/orgs/${org.id}/workspaces/${ws.id}'`, undefined, "viewer"],
    ["GET", `'/orgs/${org.id}/workspaces/${ws.id}/documents'`, undefined, "viewer"],
    ["GET", `'/orgs/${org.id}/documents/${doc.id}'`, undefined, "viewer"],
    ["POST", `'/orgs/${org.id}/workspaces'`, { name: "more" }, "editor"],
    ["POST", `'/orgs/${org.id}/workspaces/${ws.id}/documents'`, { title: "more" }, "editor"],
    ["GET", `'/orgs/${org.id}/usage'`, undefined, "owner"],
    ["GET", `'/orgs/${org.id}/audit'`, undefined, "owner"],
    ["PUT", `'/orgs/${org.id}/members'`, { email: who.viewer, role: "viewer" }, "owner"], // unchanged: the matrix must not move roles
    ["POST", `'/documents/${doc.id}/session'`, undefined, "viewer"],
    ["GET", `'/documents/${doc.id}/runs/${run.id}'`, undefined, "viewer"],
    ["POST", `'/documents/${doc.id}/runs'`, { instruction: "again" }, "editor"], // 409 for the allowed: a run is unfinished
    ["POST", `'/documents/${doc.id}/runs/${run.id}/cancel'`, undefined, "editor"],
    ["POST", `'/documents/${doc.id}/preview'`, undefined, "viewer"],
    ["GET", `'/documents/${doc.id}/preview'`, undefined, "viewer"],
    ["GET", `'/documents/${doc.id}/conflict'`, undefined, "viewer"],
    ["POST", `'/documents/${doc.id}/ship'`, undefined, "editor"],
    ["GET", `'/documents/${doc.id}/ship'`, undefined, "viewer"],
    // E8.3 (F25): the owner shares with the stranger (after every stranger row) and revokes it again. E10.8: and lists the shares.
    ["GET", `'/documents/${doc.id}/shares'`, undefined, "owner"],
    ["PUT", `'/documents/${doc.id}/shares'`, { email: who.stranger, role: "viewer" }, "owner"],
    ["DELETE", `'/documents/${doc.id}/shares/${strangerId}'`, undefined, "owner"],
  ];
  const rank: Record<Role, number> = { viewer: 0, editor: 1, owner: 2 };
  // Least privileged first: a denied write must be refused BEFORE it does anything the next row would see.
  for (const as of ["stranger", "viewer", "editor", "owner"] as const) {
    for (const [method, path, body, least] of routes) {
      const { status, json } = await call(as, method, path, body);
      const label = `${as} ${method} ${path}`;
      if (as === "stranger") {
        expect(status, label).toBe(404);
        expect(ErrorBody.parse(json).error, label).toBe("not_found");
      } else if (rank[as] < rank[least]) {
        expect(status, label).toBe(403);
        expect(ErrorBody.parse(json).error, label).toBe("forbidden");
      } else {
        expect([200, 201, 204, 409], label).toContain(status);
      }
    }
  }
  // The viewer's and editor's refused writes left nothing behind: only the owner's and editor's own creations.
  const workspaces = (await call("owner", "GET", `'/orgs/${org.id}/workspaces'`)).json as { items: unknown[] };
  expect(workspaces.items).toHaveLength(3); // the first, the editor's, the owner's
});
```

FROM apps/api/src/app.ts

```
// F24: only an owner changes roles, their own included (the last owner cannot step down: the org would have
// none). A new member is added the same way: the api has no email to invite with, so it takes a user who signed up.
// The change is committed BEFORE it is announced, so a sync node that hears of it reads the new role.
org.put("/members", need("owner"), async (c) => {
  const member = await c.var.scope.setMember({ ...(await body(c, SetMemberBody)), by: c.var.user.id });
  if (member === "no_user") return notFound(c);
  if (member === "last_owner") return fail(c, 409, "last_owner");
  if (member === "forbidden") return fail(c, 403, "forbidden"); // demoted while this request waited for the lock
  await announce(c, { orgId: c.var.org.id, userId: member.userId });
  return c.json(member);
});
```

FROM packages/db/src/index.ts

```
setMember: async ({ email, role, by }) => {
  if (!orgExists || (by !== undefined && !isId(by))) throw new Error("cannot set a member: invalid org or actor id");
  const input = { email: User.shape.email.parse(email), role: Role.parse(role) };
  // One change at a time per org: two owners demoting each other at once must not both see the other
  // still an owner (under read committed, each statement would) and leave the org with none. The actor's own role
  // is read again under the lock: the route's check ran before it, and an owner demoted since must not act.
  return inTurn(`noon:members:${orgId}`, async (client) => {
    const found = z.object({ id: z.string(), role: Role.nullable(), owners: count, actor: Role.nullable() })
    }).optional().parse(await client.query<QueryResultRow>(
      "select u.id, m.role, (select count(*) from memberships where org_id = $1 and role = 'owner') as owners, " +
      "(select role from memberships where org_id = $1 and user_id = $3) as actor " +
    ));
```

```
FROM packages/db/src/index.ts
```

`setMember` is the one place in the epic that needs a transaction rather than a statement, and the reason is a race the types cannot see. Rule: the last owner cannot step down, or the org has nobody who can change roles. Check: count the owners, refuse if the caller is the last. Race: two owners demote *each other* at the same moment; under READ COMMITTED each statement sees the other still an owner, both checks pass, and the org ends with none. There is no single row to lock (the rule is about a *set* of rows), so the transaction takes a Postgres *advisory lock* named after the org: `pg_advisory_xact_lock(hashtext('noon:members:<orgId>'))`, held until commit, so the second change waits for the first and then sees its result. The integration test fires both demotions with `Promise.all` and asserts `[200, 403]`: the caller's own role is read again under the lock, so the second change finds its caller no longer an owner. The audit row is written in the same transaction, and only when the role really changed.

Noon MVP handbook · lesson-8 · 14/39

Shares (F25). An owner shares one document with someone outside the org, at editor or viewer, never owner. The first design question was where the row lives, and the answer is *not in memberships*: a share must not make anyone a member, and every org route, plus `GET /orgs`'s keyset paging over the orgs a user belongs to, joins `memberships` and would have shown the org to the outsider. So `document_shares` is a table of its own, keyed by document and user, tied to the document's org by a composite foreign key so that a share can never point at a document of another org.

FROM `packages/db/migrations/0016_document_shares.sql`

```
-- E8.3 (F25): one document shared with a user outside its org, at editor or viewer (never owner: a share opens a
-- document, never the org). A table of its own, never rows in memberships: a share must not make anyone a member
-- (every org route, and GET /orgs's paging over orgs join memberships, stay exactly as they were).
-- Revoking deletes the row. The composite key ties the share to the document's org, and dies with the document.
create table document_shares (
  org_id      uuid not null,
  document_id uuid not null,
  user_id     uuid not null references users (id) on delete cascade,
  role        text not null check (role in ('editor', 'viewer')),
  created_at  timestampz not null default now(),
  primary key (document_id, user_id),
  foreign key (document_id, org_id) references documents (id, org_id) on delete cascade
);
-- Backs the users foreign key when a user is deleted.
create index document_shares_user_id on document_shares (user_id);
```

FROM `packages/db/src/index.ts`

```
type ShareRole = z.infer<typeof ShareBody>["role"];
/**
 * E8.3: the role of user `<n>` on document `d`: their role in its org or their share of it, whichever is higher; null
 * when they have neither. A scalar subquery: still one round trip per request (the E1.4 finding), two primary-key probes.
 */
const accessOf = (userParam: string): string =>
  "(select role from (select role from memberships where org_id = d.org_id and user_id = " + userParam +
  " union all select role from document_shares where document_id = d.id and user_id = " + userParam +
  ") a order by array_position(array['viewer', 'editor', 'owner'], role) desc limit 1)";
```

FROM `packages/db/src/index.ts`

```
getDocumentForMember: async (documentId, userId) =>
  isId(documentId) && isId(userId)
    ? one(z.looseObject({ role: Role }).transform((r) => ({ document: DocumentRow.parse(r), role: r.role })), `select * from (select
d.*, ${accessOf("$2")} as role from documents d where d.id = $1) x where role is not null`, [documentId, userId])
    : undefined,

roleIn: async (orgId, documentId, userId) =>
  isId(orgId) && isId(documentId) && isId(userId)
    ? (await one(z.object({ role: Role.nullable() })), `select ${accessOf("$3")} as role from documents d where d.org_id = $1 and d.id
= $2`, [orgId, documentId, userId])?.role ?? undefined
    : undefined,
```

`accessOf` is a scalar subquery spliced into the document lookup, and it answers "the higher of the caller's org role and their share" in SQL: a `union all` of the two candidate roles, ordered by their position in the array `['viewer', 'editor', 'owner']` descending, `limit 1`. That is "max by enum order" without a numeric column, and it costs two primary-key probes inside the one round trip the document routes already made (the E1.4 rule: one lookup per request). A member who is also shared a document at a higher role gets the higher one; revoking the share leaves their org role; an outsider with no share gets `null`, which the outer `where role is not null` turns into no row, and no row is 404.

FROM `apps/api/src/app.ts`

```
// Everything under /documents/:id: the path names no org, so the lookup itself is access-filtered (a member of the
// org, or someone the document is shared with, E8.3), and the same row carries the caller's role for each route's
// 'need' (E8.2). Neither: 404, as for an org. So a revoked share's next /session is a 404, and its peer gives up.
// E9.6: minting a sync session has its own, tighter limit per user, taken BEFORE the document is looked up. A
// collaborator whose share was revoked gets 404 here; a peer that keeps asking regardless is soon refused with a
// Retry-After (and a real client backs off), on top of its per-user limit.
app.use("/documents/:id/session", createMiddleware<{ Variables: { user: User } }>(async (c, next) => {
  if (c.req.method !== "POST") return next();
  const verdict = await db.take(`mint:${c.var.user.id}`, limits.mint);
  return verdict.ok ? next() : limited(c, verdict.retryAfterSeconds);
}));

const document = new Hono<{ Variables: { user: User; doc: Document; role: Role } }>();
document.use(async (c, next) => {
  const found = await db.getDocumentForMember(c.req.param("id") ?? "", c.var.user.id);
  if (!found) return notFound(c);
  c.set("doc", found.document);
  c.set("role", found.role);
```

```
    await next();
  });
}
```

FROM `apps/api/src/app.ts`

```
// F25: an owner of the document's org shares it with someone outside the org (or changes the share), and revokes it.
// Committed BEFORE it is announced, like a role change: every sync node re-reads that user's live sessions, and a
// revoked one is closed. A token minted before the revoke is refused at the upgrade, which reads the row too.
document.put("/shares", need("owner"), async (c) => {
  const { email, role } = await body(c, ShareBody);
  const member = await db.forOrg(c.var.doc.orgId).share({ documentId: c.var.doc.id, email, role, by: c.var.user.id });
  if (!member) return notFound(c); // nobody has that email (the api has no email to invite with)
  await announce(c, { orgId: c.var.doc.orgId, userId: member.userId });
  return c.json(member);
});
document.delete("/shares/:userId", need("owner"), async (c) => {
  const userId = c.req.param("userId");
  if (!(await db.forOrg(c.var.doc.orgId).unshare(c.var.doc.id, userId, c.var.user.id))) return notFound(c);
  await announce(c, { orgId: c.var.doc.orgId, userId });
  return c.body(null, 204);
});
```

FROM `packages/db/src/index.ts`

```
share: async ({ documentId, email, role, by }) => {
  const input = ShareBody.parse({ email, role });
  if (!orgExists || !isId(documentId) || (by !== undefined && !isId(by))) return undefined;
  return one(
    MemberRow,
    // insert ... select: a row only when the document is this org's and the user exists; audited in the same statement.
    "with s as (insert into document_shares (org_id, document_id, user_id, role) select d.org_id, d.id, u.id, $4 from documents d,
users u " +
    "where d.org_id = $1 and d.id = $2 and u.email = lower($3) on conflict (document_id, user_id) do update set role =
excluded.role returning user_id, role), " +
    'a as (insert into audit_log ${AUDIT_COLUMNS} select $1, ${factorOf("$5")}, 'share_granted', $2, jsonb_build_object('email',
u.email, 'role', s.role) from s join users u on u.id = s.user_id) ` +
    "select u.id, u.email, u.name, s.role from s join users u on u.id = s.user_id",
    [orgId, documentId, input.email, input.role, by ?? null],
  );
},
unshare: async (documentId, userId, by) =>
  orgExists && isId(documentId) && isId(userId) && (by === undefined || isId(by)) &&
  (await pool.query(
    // The audit row is inserted FROM the deleted row: no share, no row, and the count says which.
    `with d as (delete from document_shares where org_id = $1 and document_id = $2 and user_id = $3 returning org_id, document_id,
user_id) ` +
    `insert into audit_log ${AUDIT_COLUMNS} select d.org_id, ${factorOf("$4")}, 'share_revoked', d.document_id,
jsonb_build_object('email', u.email) from d join users u on u.id = d.user_id`,
    [orgId, documentId, userId, by ?? null],
  )).rowCount === 1,
```

Both share statements are one `insert ... select` or `delete ... returning` with the audit row in a CTE of the same statement. `share` selects from `documents` and `users` together, so a row is written only when the document is this org's *and* the email exists; either missing is zero rows, which the route answers as 404. `unshare` inserts the audit row *from the deleted row*, so revoking a share that does not exist deletes nothing, records nothing, and the row count says which happened. Every write is committed before it is announced (section 4): a sync node that hears of a change and reads the row must see the new one.

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>createMiddleware<{ Variables: { role: Role } }>: typed request context</code>	<code>SecurityContextHolder</code> , or request attributes (untyped)	<code>request.user</code> , an attribute set by middleware
<code>new WeakMap<object, Role>(): metadata on a function without touching it</code>	annotations read by reflection	function attributes, or a decorator registry
<code>RANK: Record<Role, number> and includes(role, need)</code>	<code>RoleHierarchyImpl: OWNER > EDITOR > VIEWER</code>	no hierarchy in Django's permissions; groups, or an <code>IntEnum</code>
<code>c.var.role: typed by the middleware's generic</code>	<code>@AuthenticationPrincipal</code>	<code>request.user.role</code>
<code>pg_advisory_xact_lock(hashtext(\$1))</code> around a set-level rule	the same SQL; or <code>@Lock(PESSIMISTIC_WRITE)</code> on a parent row when there is one	the same SQL; <code>select_for_update</code> when a row exists to lock
<code>array_position(array['viewer', 'editor', 'owner'], role): an enum's order, in SQL</code>	a <code>CASE</code> expression, or a rank column	the same SQL
<code>foreign key (document_id, org_id) references documents (id, org_id): a composite key that pins the tenant</code>	the same DDL; JPA needs an <code>@IdClass</code>	the same DDL; Django cannot declare it, use a migration

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The role in the sync token was rejected (the E1.5 decision, kept).

It would have saved the sync server a read. But a token is checked once, at connect, and a socket then lives for hours: a role in it is stale the moment an owner changes anything, and a demoted editor keeps editing until they reconnect. The token says *who*; Postgres says *what*, at the moment of asking. Section 4 is how the sync server asks.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Shares as rows in memberships were rejected.

A share with a flag on the membership row would have been one table and one join. It would also have made every org route see the outsider as a member, listed the org in their `GET /orgs`, and let `setMember` promote a share to owner by accident. A table with its own key and a role constraint that excludes `owner` makes the wrong things impossible to write.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A second query for the role was rejected.

Reading the membership for the 404 and the role for the 403 in two statements is the obvious layering, and it doubles the round trips of every request for nothing: the membership row *is* the role. E1.4 had already found that identity must be one read; E8.2 keeps authorisation inside the same one, and E8.3 keeps the share lookup inside it too, as a subquery.

The principle: authorise where you fetch. A check in the handler can be forgotten; a check in the query that returns the data cannot be bypassed by any route that uses the query, and a table of routes with their guards, compared by a test, catches the route that used neither. Postgres row-level security is the same idea one layer down, and the natural next step if the api ever grows a second way to read the tables.

4 · Roles in the room: read before the upgrade, re-read at the op's turn, and revocation that beats the token

REST is easy: every request reads the row. A WebSocket is the hard case, because it is authenticated once and then lives for hours, and F24 demands that a role change reach an *open* session within ten seconds. The sync server never trusts the token for what someone may do (section 3's case study), so it reads the role itself, from Postgres, at three moments: **before the upgrade**, when the token arrives; **at the join**, once the socket is registered; and **on every announcement and every 30 seconds**, for as long as the socket lives. The result of a read is one boolean on the peer, `mayEdit`, and the room checks it when an op's *turn* comes.

FROM apps/sync/src/server.ts

```
/**
 * E8.2 (F24): a user's role on the document, if it is that org's and they are a member or it is shared with them
 * (db.roleIn, E8.3). Asked at the upgrade, when a peer joins and on every `recheck`: a viewer's ops are refused, and
 * someone with no access is refused a socket (401) or closed out (4404). The token says only who someone is, never
 * what they may do. Without it every peer may edit (tests without a database).
 */
roles?: (orgId: string, documentId: string, userId: string) => Promise<Role | undefined>;
/**
 * E8.3 (F25): every live session's role is read again this often, whatever was announced: the backstop for an
 * announcement the api could not publish (it tries once), so a revoked share cannot stay open for good.
 */
sweepMs?: number;
};
```

FROM apps/sync/src/room.ts

```
/** E8.2 (F24): may this peer change the document? False for a viewer. The server may flip it at any moment (a role change). */
mayEdit: boolean;
};
```

FROM apps/sync/src/room.ts

```
if (!peers.has(peer)) return; // dropped while this op waited in the queue
// F24: asked when the op's TURN comes, not when it arrived: an op queued before a demotion is refused too.
if (!peer.mayEdit) {
  refuse("forbidden");
  return;
}
```

"At its turn, not when it arrived" is the sentence to remember. The room is a queue (lesson 2): an op can wait behind others, and a demotion can land while it waits. Checking `mayEdit` when the op is *received* would let every op already queued through; checking at the head of the queue, one line before the dedupe and the journal, refuses them all. The refusal is `forbidden`, a reason the client turns into "You can view this document but not edit it." and undoes on its own canvas; the viewer stays connected and keeps seeing everyone else's ops and presence.

FROM apps/sync/src/room.test.ts

```
// E8.2 (F24): a viewer sees everything and changes nothing.
test("a viewer's op is refused as forbidden, applied to nothing and heard by nobody, while it still sees the others' ops and presence",
  async () => {
    const journal = memoryJournal();
    const room = createRoom({ doc: emptyDoc(), manifest, journal });
    const [editor, viewer] = [peer("e"), { ...peer("v"), mayEdit: false }];
    room.join(editor); room.join(viewer);
    const refused = clientOp(add("from-viewer"));
    await room.submit(viewer, refused);
    expect(rejects(viewer)).toEqual([ { type: "rejected", opId: refused.opId, reason: "forbidden" } ]);
    expect(room.doc.nodes["from-viewer"]).toBeUndefined();
    expect(journal.rows).toEqual([]);
    expect(room.seq).toBe(0);
    expect(editor.inbox.filter((m) => m.type !== "welcome")).toEqual([]); // nobody heard of it

    await room.submit(editor, clientOp(add("from-editor")));
    expect(ops(viewer).map((m) => m.seq)).toEqual([1]); // it sees the edit live
    room.presence(editor, { cursor: { x: 0.5, y: 0.5 }, selection: null });
    expect(viewer.inbox.some((m) => m.type === "presence")).toBe(true);
  });
```

The first read, before the upgrade. E8.3 moved it there, and the reason is the whole of F25's hard clause: *a token minted before the revoke must not get them back in*. The sync token lives 60 seconds and its signature is valid for all of them. A revoked outsider who kept one (their client did, to reconnect) would present a perfectly signed token, and a server that admitted on the signature alone would open the socket and only then discover, at the join, that there is no access, and close it with 4404. That is a socket that should never have existed, and, for a client that reconnects, a way to keep reopening one for a minute. So `admits` reads the row while the request is still HTTP, and a revoked peer's reconnect is a plain 401, indistinguishable from any other bad token. Postgres not answering is not a no (the join asks again and says "try again", 4503).

FROM apps/sync/src/server.ts

```
/**
 * E8.3 (F25): may the token's holder still open the document? A token lives 60 s: one minted before a revoke (or
 * before a demotion out of the org) must not open a socket, so a valid signature alone is not enough. Asked BEFORE
 * the upgrade, so a revoked peer's reconnect is a plain 401, like any other token that is no good. Postgres not
 * answering is not a no: serve() asks again, and says "try again" (4503). A revoke landing after this read and
 * before serve() registers the socket is still caught: serve() reads once more, after registering it.
 * ponytail: two reads per connect; ceiling: none that matters (primary-key probes); upgrade: carry this answer into
```

```

    * serve() with a "changed meanwhile" flag.
    */
    async function admits(claims: { userId: string; orgId: string; actor: { kind: "user" | "agent" | "git" } }, documentId: string):
    Promise<boolean> {
        if (!roles || claims.actor.kind === "git") return true;
        try {
            return (await roles(claims.orgId, documentId, claims.userId)) !== undefined;
        } catch {
            return true;
        }
    }
}

```

FROM `apps/sync/src/server.ts`

```

// Every socket past the upgrade, from BEFORE its role is first read: a change published while that read is in
// flight still finds it. 'asked'/'applied' number the reads, so an older answer never replaces a newer one.
type Connected = { peer: Peer; ws: WebSocket; documentId: string; asked: number; applied: number };
const connected = new Set<Connected>();

```

FROM `apps/sync/src/server.ts`

```

// E8.2 (F24): who may be here and edit is Postgres's answer, asked BEFORE any room is opened for them.
const entry: Connected = { peer, ws, documentId, asked: 0, applied: 0 };
connected.add(entry);
ws.on("close", () => { connected.delete(entry); });
try {
    if (!(await applyRole(entry))) return;
} catch {
    ws.close(CLOSE.unavailable, "unavailable"); // "try again", as when the document cannot be loaded
    return;
}

```

FROM `apps/sync/src/server.ts`

```

/**
 * Reads what this peer may do, and applies it. False: it is no member (any more), and was closed out. Rejects when
 * Postgres did not answer. The git peer is the service itself (its id names a commit event, not a person); an AI
 * run is its creator, so it can do what they can.
 */
async function applyRole(entry: Connected): Promise<boolean> {
    const { peer, ws, documentId } = entry;
    if (!roles || peer.actor.kind === "git") {
        peer.mayEdit = true;
        return true;
    }
    const asked = ++entry.asked;
    const role = await roles(peer.session.orgId, documentId, peer.session.userId);
    if (asked < entry.applied) return ws.readyState === ws.OPEN; // a newer read already decided
    entry.applied = asked;
    if (role === undefined) {
        // The same answer a stranger's token gets: the document is not there for them.
        ws.close(CLOSE.documentNotFound, "cannot_open_document");
        return false;
    }
    peer.mayEdit = role !== "viewer";
    return true;
}

async function recheck(change: { orgId: string; userId: string } | "all"): Promise<void> {
    const due = [...connected].filter(({ peer }) => change === "all" || (peer.session.orgId === change.orgId && peer.session.userId === change.userId));
    await Promise.all(due.map(async (entry) => {
        // Until Postgres answers or the socket goes: a demotion must not be lost to one failed read.
        while (!closing && connected.has(entry)) {
            try {
                await applyRole(entry);
                return;
            } catch (err) {
                log(entry.documentId, 'role not read again, retrying: $${err instanceof Error ? err.message : "unknown"}');
                await new Promise((resolve) => setTimeout(resolve, recoverMs).unref());
            }
        }
    }));
}

```

Read `applyRole` and `recheck` with lesson 7's eye for "which answer is newer". A socket is added to `connected` *before* its role is first read, so an announcement published while that read is in flight still finds it and reads again. Now two reads are in flight for one socket and can return in either order; `asked` and `applied` number them, and an older answer that arrives after a newer one is dropped (`asked < entry.applied`). Without that, a

revoke could be undone by a read that started before it: the race test in `role-change.int` holds the first read's answer, revokes, lets the announcement's read land, then releases the old one, and asserts the peer never joins. A read that fails is retried until Postgres answers or the socket goes: a demotion must not be lost to one dropped connection. And the git peer is never asked: it is the service itself, not a person.

The announcement is a hint. The api commits the change, then publishes `{ orgId, userId }` on one Redis channel; every sync node re-reads the role of the live sessions that pair concerns. The message carries no role, on purpose: a forged or stale message can only cause a re-read, never grant anything. Pub/sub keeps nothing for a subscriber that was away, so a node whose connection comes back is told `"all"` and re-reads every live session. And because the api announces once, best effort (a ponytail E8.2 named), E8.3 found that a revoke could stay open for good if Redis was away at the api when it happened; the fix is a *sweep*: every node re-reads all its live sessions every 30 seconds, never two sweeps at once. REST is never behind, because it reads the row per request; only the sockets need the push.

FROM `packages/lease/src/access.ts`

```
/**
 * E8.2 (F24): "this user's access to this org changed". A HINT, never the new role: every sync node reads the
 * role itself, from Postgres, so a forged or stale message can only cause a re-read, never grant anything.
 * Pub/sub delivers to whoever is subscribed at that moment and keeps nothing, so a subscriber that was
 * disconnected is told "all": it may have missed changes, and re-reads every live session's role.
 */
const AccessChange = z.strictObject({ orgId: z.uuid(), userId: z.uuid() });
type AccessChange = z.infer<typeof AccessChange>;
```

FROM `packages/lease/src/access.ts`

```
/**
 * A sync node's half. `onChange` hears every change, and "all" each time the connection is (re)established, the
 * first time included. ioredis reconnects for ever and subscribes again by itself; with Redis away at startup the
 * SUBSCRIBE waits in its offline queue. `subscribed` resolves once Redis has it (tests wait for it).
 * ponytail: "all" fires as the connection is ready, as the SUBSCRIBE is sent, not once Redis confirmed it; ceiling:
 * a change published in that round trip, after the re-read began, is missed; upgrade: re-read on the subscribe reply.
 */
export function accessSubscriber({ redisUrl, prefix = "", onChange }: { redisUrl: string; prefix?: string; onChange: (change: AccessChange | "all") => void }): { subscribed: Promise<void>; close(): Promise<void> } {
  const redis = new Redis(redisUrl);
  redis.on("error", (err: unknown) => { warn(err instanceof Error ? err.message || err.name : "unknown"); });
  const channel = channelOf(prefix);
  redis.on("message", (from: string, text: string) => {
    if (from !== channel) return;
    let parsed;
    try {
      parsed = AccessChange.safeParse(JSON.parse(text));
    } catch {
      parsed = undefined;
    }
    if (parsed?.success) onChange(parsed.data);
    else warn("an access change that is not one was ignored");
  });
  redis.on("ready", () => { onChange("all"); });
  const subscribed = redis.subscribe(channel).then(() => undefined);
  subscribed.catch(() => undefined); // closed before it connected: nothing to report
  return {
    subscribed,
    async close() {
      await redis.quit().catch(() => { redis.disconnect(); });
    },
  };
}
```

FROM `apps/sync/src/main.ts`

```
// E8.2 (F24): the api announces role changes on Redis; every live session they concern reads its role again.
const access = accessSubscriber({ redisUrl: config.lease.redisUrl, onChange: (change) => void server.recheck(change) });
```

FROM `apps/api/src/app.ts`

```
// ponytail: announced once, best effort; ceiling: with Redis away here (but not at the sync nodes, which re-read
// everything when their own link comes back) open sessions keep the old access until the sync nodes' sweep (30 s),
// or until they reconnect; upgrade: an outbox row the api retries. REST routes, /session and the sync upgrade read
// the access on every request, so they are never behind.
const announce = async (c: Context, change: { orgId: string; userId: string }): Promise<void> => {
  await accessChanged(change).catch((err: unknown) => {
    process.stderr.write(`${JSON.stringify({ level: "warn", path: c.req.path, message: `access change not announced:
${describeError(err)}\n`});
  });
}
```



```
};
const attempt = allowAttempt ?? ((key: string) => db.take(`attempt:${createHash("sha256").update(key).digest("hex")}`, limits.attempt));
```

FROM `apps/sync/src/server.ts`

```
// ponytail: one indexed read per live session per sweep; ceiling: a few thousand sessions a node (a sweep never
// overlaps the last one); upgrade: one query for all of a node's (org, document, user) triples.
let sweeping = false;
const sweeper = roles ? setInterval(() => {
  if (sweeping) return;
  sweeping = true;
  void recheck("all").finally(() => { sweeping = false; });
}, sweepMs) : undefined;
sweeper?.unref();
```

FROM `apps/sync/src/role-change.int.test.ts`

```
test("an owner demotes an editor with the document open: its next op is refused within 10 s; promoted again, it edits within 10 s", async () => {
  const { org, doc } = await world({ editor: "editor" });
  const editor = browser("editor", doc.id);
  try {
    await until(() => editor.status === "live", "editor live");
    const first = editor.submit(add());
    expect(first.ok && (await first.settled)).toMatchObject({ ok: true });

    /** Submits an op every 100 ms until one settles as `want`; returns how long that took. */
    async function msUntil(want: "forbidden" | "accepted"): Promise<number> {
      const since = Date.now();
      for (;;) {
        const tried = editor.submit(add());
        const outcome = tried.ok ? await tried.settled : tried;
        if (want === "forbidden" ? !outcome.ok && outcome.reason === "forbidden" : outcome.ok) return Date.now() - since;
        if (Date.now() - since > 10_000) throw new Error(`no ${want} op within 10 s`);
        await new Promise((resolve) => setTimeout(resolve, 100));
      }
    }
    await setRole(org.id, "editor", "viewer");
    expect(await msUntil("forbidden")).toBeLessThan(10_000);
    expect(editor.status).toBe("live"); // the SAME connection: no reconnect was needed for the change to apply
    await setRole(org.id, "editor", "editor");
    expect(await msUntil("accepted")).toBeLessThan(10_000);
  } finally {
    editor.close();
  }
});
```

Every way back, shut. Revocation closes the live socket with 4404, which peer-client treats as *fatal*: no reconnect. It is the same code a stranger's token would get at the join, so a revoked person learns nothing they did not know. A peer that reconnects by itself asks `/session`, whose document middleware reads the row and answers 404; peer-client reads a `null` session as "give up" (`no_session`). And the token minted before the revoke meets `admits`, above. The e2e spec does all three from a real second browser, and asserts the guest's canvas is gone and its status line with it.

FROM `packages/peer-client/src/peer.ts`

```
// Close codes that reconnecting cannot cure (apps/sync/src/server.ts): a message the server could
// not accept (our bug), no such document, a corrupt document, a frame over the size limit.
const FATAL_CLOSE_CODES = new Set([4400, 4404, 4500, 1009]);
```

FROM `apps/sync/src/role-change.int.test.ts`

```
test("shared with an outside user, they edit live; revoked, their session closes within 10 s, no op reaches them after, and every way back is refused", async () => {
  const { doc } = await world({});
  const shared = await shareWithOutsider(doc.id);
  const owner = browser("owner", doc.id);
  const outsider = browser("outsider", doc.id);
  try {
    await until(() => owner.status === "live" && outsider.status === "live", "both live");
    const edit = outsider.submit(add());
    expect(edit.ok && (await edit.settled)).toMatchObject({ ok: true });
    const early = await mint("outsider", doc.id);

    const since = Date.now();
    await revoke(doc.id, shared.userId);
    await until(() => outsider.status === "closed", "the outsider's session is closed", 10_000);
    expect(Date.now() - since).toBeLessThan(10_000);
    expect(outsider.closedBecause).toBe("4404"); // fatal for the client: it does not come back by itself
  }
});
```

```

const seen = outsider.seq;
const after = owner.submit(add());
expect(after.ok && (await after.settled)).toMatchObject({ ok: true });
await new Promise((resolve) => setTimeout(resolve, 100));
expect(outsider.seq).toBe(seen);
expect(sync.peerCount(doc.id)).toBe(1); // only the owner is in the room

// Every way back is shut: a new session, the token minted before the revoke, and a peer reconnecting on its own.
expect((await call("outsider", "POST", `/documents/${doc.id}/session`)).status).toBe(404);
expect(await upgradeStatus(`${sync.url}/documents/${doc.id}`, ["noon.v1", "early"], "a token minted before the revoke").toBe(401);
const again = browser("outsider", doc.id);
await until(() => again.status === "closed", "a reconnecting peer gives up");
expect(again.closedBecause).toBe("no_session");
again.close();
} finally {
  owner.close();
  outsider.close();
}
});

```

FROM `apps/sync/src/role-change.int.test.ts`

```

// The revoke lands while the peer is on its way in, its access already read as "shared": at the upgrade's own read
// (the socket is not registered yet, so the announcement finds nobody), or at the read serve() makes once it is (the
// announcement makes the node read again, and that newer answer wins over the late one).
for (const [nth, which] of [[1, "the upgrade's read"], [2, "the read after the socket is registered"]]) as const {
  test(`a revoke landing during ${which}, with a token minted before it, keeps the peer out: it never joins, and is closed (4404)`, async
  () => {
    const { doc } = await world({});
    const shared = await shareWithOutsider(doc.id);
    const token = await mint("outsider", doc.id);
    let reads = 0;
    const held = Promise.withResolvers<undefined>();
    const release = Promise.withResolvers<undefined>();
    afterRead = async (userId) => {
      if (userId !== shared.userId || ++reads !== nth) return;
      held.resolve(undefined);
      await release.promise;
    };
    const statuses: string[] = [];
    const peer = connectPeer({
      manifest,
      retryMs: { min: 20, max: 100 },
      onStatus: (status) => { statuses.push(status); },
      session: () => Promise.resolve({ wsUrl: `${sync.url}/documents/${doc.id}`, token }),
    });
    try {
      await held.promise; // read as still shared; the answer is held on its way
      await revoke(doc.id, shared.userId); // committed and announced meanwhile
      if (nth === 2) await until(() => reads > 2, "the announcement made the node read again");
      release.resolve(undefined);
      await until(() => peer.status === "closed", "the peer is closed out");
      expect(peer.closedBecause).toBe("4404");
      expect(statuses).not.toContain("live"); // no welcome: it never saw the document
      expect(sync.peerCount(doc.id)).toBe(0);
    } finally {
      release.resolve(undefined);
      afterRead = undefined;
      peer.close();
    }
  });
}

```

TYPESCRIPT (THIS REPO)	JAVA	PYTHON
<code>Promise.withResolvers()</code> : a promise and its <code>resolve</code> , to hold a read from a test	<code>CompletableFuture</code> , completed by hand	<code>asyncio.Future</code> , or an <code>asyncio.Event</code>
<code>setInterval(...).unref()</code> : a timer that does not keep the process alive	a daemon thread	not needed: the loop ends when the main task does
<code>Set<Connected></code> plus <code>asked</code> / <code>applied</code> counters	<code>ConcurrentHashMap</code> and an <code>AtomicLong</code> per entry	a dict and ints, safe under the one event loop
<code>redis.subscribe(channel)</code> and <code>redis.on("message", ...)</code> on a client of its own	Jedis <code>JedisPubSub</code> on a dedicated connection; Lettuce's <code>StatefulRedisPubSubConnection</code>	<code>redis.psubsub()</code> with <code>listen()</code> or <code>run_in_thread</code>
<code>void sync.recheck(change)</code> : fire and forget, on purpose, with the promise never rejecting	<code>CompletableFuture.runAsync</code>	<code>asyncio.create_task</code>
<code>ws.close(4404, "cannot_open_document")</code> : an application close code	<code>session.close(new CloseStatus(4404, ...))</code>	<code>await websocket.close(code=4404)</code>

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Polling the role from the room was rejected: the E1.5 review asked for push.

A read of the role every few seconds per live session would meet the ten-second budget and needs no Redis. It also scales with the number of sockets, not with the number of changes, which is the wrong axis: most sessions never see a role change. The push carries who changed; the node reads only those. The 30-second sweep then exists for the failure case only, and its cost is one primary-key probe per session per half minute.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Checking mayEdit when an op arrives was rejected: at its turn.

The first shape refused a viewer's op in `submit` , before it was queued. A demotion that landed while an editor had ops waiting let every one of them through, because they had already passed the check. Moving the check to the head of the queue, next to the other things the room asks of an op when it is finally its turn, made "an op queued before a demotion is refused too" true, and the unit test pins it.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The E8.2 ponytail was not acceptable for a revoke.

Announcing once, best effort, was fine for a demotion: the person keeps editing until they reconnect, and REST already refuses them. It was not fine for a revoked share: with Redis away at the api at that moment, an outsider would keep a live socket into a document they no longer have access to, for as long as they kept it open. The sweep closes that window at 30 seconds, and the test unshares behind the api's back and waits for the sweep to close the socket.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A revocation epoch in the token was rejected.

The textbook answer to "a token outlives a revoke" is a counter in the token that the server bumps on revoke, so an older token fails a comparison. It needs the server to store the counter per user, which is a deny list by another name, and it does nothing for the live socket, which outlives every token anyway. Reading the row before the upgrade and re-reading it on the socket answers both cases with one mechanism.

Revocation must beat every lifetime you hand out. There are three here. The sign-in cookie (7 days): a row, so revocation is a `DELETE` and the next request fails. The sync token (60 s): the row is read *before the upgrade*, so its lifetime never matters. The socket (hours): the role is re-read on every announcement and every 30 seconds, and the room checks the result at each op's turn. A design that leaves any one of the three to "wait it out" has a window exactly that long, and F25's test asks for all three to be shut at once.

5 · The audit trail: written by the action, and the database refuses to change it

An audit trail has three properties or it is a log. It is **complete**: an action that happened has a row, and an action that did not happen has none. It is **immutable**: nobody who can write the trail can rewrite it, the application included. It is **durable against deletion elsewhere**: removing the org, the user or the document must not remove the record of what they did. Each property is enforced by the database rather than by the application, and the migration says how.

FROM packages/db/migrations/0017_audit_log.sql

```
-- E8.4 (F26): the org's audit trail. Append-only, and the DATABASE says so, twice: the app's login role is never
-- granted UPDATE, DELETE or TRUNCATE on it (provisionAppRole revokes them), and a trigger refuses them to anyone else
-- who connects, the owner included. Each row is written in the same statement (or transaction) as the action it
-- records, so an action cannot commit unaudited, nor an audit row for an action that rolled back.
--
-- No foreign keys at all, on purpose (the E1.2 review): 'on delete cascade' from orgs would erase an org's trail
-- together with the org, and 'on delete set null' from users is an UPDATE. So the actor's email is copied in as it
-- was at the time, and a row may name a user, a document or an org that no longer exists.
create table audit_log (
  id          uuid primary key default gen_random_uuid(),
  org_id      uuid not null,
  -- user: a person (actor_id); git: a push to a document's branch (its commit is in detail); system: a job or
  -- change with no person behind it (a run created by a script, a role changed from a test).
  actor_kind  text not null check (actor_kind in ('user', 'git', 'system')),
  actor_id    uuid,
  actor_email text check (length(actor_email) between 3 and 320),
  action      text not null check (action in ('signed_in', 'role_changed', 'share_granted', 'share_revoked', 'run_started',
'ship_started', 'push_rejected')),
  document_id uuid,
  -- Flat text values only (the reader parses them as such); capped, as an instruction alone may be 4000 characters.
  detail      jsonb not null default '{}' check (jsonb_typeof(detail) = 'object' and octet_length(detail::text) <= 20000),
  created_at  timestamptz not null default now(),
  check ((actor_kind = 'user') = (actor_id is not null))
);
-- The audit view pages an org's rows newest first.
create index audit_log_org on audit_log (org_id, created_at, id);

create function audit_log_append_only() returns trigger language plpgsql as $$
begin
  raise exception 'audit_log is append-only: % refused', tg_op;
end
$$;
create trigger audit_log_no_change before update or delete on audit_log for each row execute function audit_log_append_only();
create trigger audit_log_no_truncate before truncate on audit_log for each statement execute function audit_log_append_only();
```

Complete: the row is written by the action's own statement. Every audited action in the db package inserts into `audit_log` either in a CTE of the statement that performs it or inside the same transaction, never after it returns. Then an action cannot commit without its row (a crash between the two would otherwise lose the record), and a row cannot outlive an action that rolled back (a refused `last_owner`, a run that hit `run_in_progress`, a wrong password, a share of nobody: nothing is written). `actorOf` copies the actor's email in as it is *now*, because the row must still read correctly when the user is gone.

FROM packages/db/src/index.ts

```
// E8.4 (F26): every audited action inserts into audit_log IN ITS OWN STATEMENT (a CTE) or transaction, never after it.
const AUDIT_COLUMNS = "(org_id, actor_kind, actor_id, actor_email, action, document_id, detail)";
/** The actor columns for the user id in parameter `p` (null: the system acted), with their email as it is now. */
const actorOf = (p: string): string =>
  `case when ${p}::uuid is null then 'system' else 'user' end, ${p}::uuid, (select email from users where id = ${p}::uuid)`;
const AuditRow = z
  .object({ id: z.string(), org_id: z.string(), actor_kind: z.string(), actor_id: z.string().nullable(), actor_email:
z.string().nullable(), action: z.string(), document_id: z.string().nullable(), detail: z.unknown(), created_at: timestamp })
  .transform((r): AuditEntry =>
    AuditEntry.parse({ id: r.id, orgId: r.org_id, actor: { kind: r.actor_kind, id: r.actor_id, email: r.actor_email }, action: r.action,
documentId: r.document_id, detail: r.detail, at: r.created_at }));
```

FROM packages/db/src/index.ts

```
/** One statement: the run, if the document is this org's and has no unfinished run, and its audit row. */
const insertRun = async (documentId: string, input: z.infer<typeof CreateRunBody>, createdBy: string | undefined, via: Pool |
PoolClient): Promise<Run | "busy" | undefined> => {
  try {
    return await one(
      RunRow,
      // insert ... select: the row is only created if the document exists in this org. Audited in the same statement.
      "with j as (insert into jobs (org_id, document_id, queue, input, created_by) select d.org_id, d.id, 'ai', $3, $4 from
documents d where d.org_id = $1 and d.id = $2 returning *), " +
      `a as (insert into audit_log ${AUDIT_COLUMNS} select j.org_id, ${actorOf("j.created_by")}, 'run_started', j.document_id,
jsonb_build_object('run', j.id::text, 'instruction', j.input ->> 'instruction') from j) ` +
```

```

        "select * from j",
        [orgId, documentId, JSON.stringify(input), createdBy ?? null],
        via,
    );
} catch (err) {
    // The unique index IS the check: "count, then insert" would let two requests at the same moment both in.
    if (err instanceof Error && "constraint" in err && err.constraint === "jobs_one_unfinished_run_per_document") return "busy";
    throw err;
}
};

```

FROM packages/db/src/index.ts

```

async recordConflict(documentId, input) {
    if (!isId(documentId)) return;
    const c = ConflictInput.parse(input);
    // Selected from documents: a document deleted meanwhile inserts nothing instead of failing on the foreign key.
    // The rejected push goes into the document's org's audit trail in the same statement (F26). ponytail: the actor
    // is "git" and the commit, not the person who pushed; ceiling: the trail cannot name them; upgrade: keep the
    // webhook's pusher on git_events and copy it here.
    await pool.query(
        "with c as (insert into document_conflicts (document_id, commit_sha, file, reason, detail) select id, $2, $3, $4, $5 from documents where id = $1 " +
        "on conflict (document_id) do update set commit_sha = excluded.commit_sha, file = excluded.file, reason = excluded.reason, detail = excluded.detail, created_at = now() returning document_id) " +
        'insert into audit_log ${AUDIT_COLUMNS} select d.org_id, \'git\', null, null, \'push_rejected\', d.id, ' +
        jsonb_build_object('commit', $2::text, 'file', $3::text, 'reason', $4::text) from c join documents d on d.id = c.document_id',
        [documentId, c.commit, c.file, c.reason, c.detail],
    );
},

```

Section 2 showed `startSession` writing one sign-in row per org the user belongs to (an org sees its members' sign-ins and nobody else's), section 3 showed `setMember` writing only when the role really changed, inside its transaction, and `share` and `unshare` writing from the row they wrote or deleted. `createRun` keeps the instruction, exactly as typed, markup and all, because the trail must say what was asked; the reader's job is to render it as text. `recordConflict`, which the worker's git peer calls, writes the rejected push with the actor `git` (a ponytail: not the person who pushed; the webhook's pusher on `git_events` is the upgrade).

Immutable: the database says so, twice. The app connects as `noon_app`, a role that `provisionAppRole` creates with data access only. It gets a blanket `GRANT select, insert, update, delete on all tables`, and `alter default privileges` so that new tables get the same; then, *after* the grant and on every provisioning, `UPDATE`, `DELETE` and `TRUNCATE` on `audit_log` are revoked. "On every provisioning" is the part that matters: a grant someone made by hand is gone after the next deploy, and the test grants them and re-provisions to prove it. The trigger is the second layer, for everyone the grants do not reach: the table's owner (grants do not bind an owner), and any role that was widened between deploys. It raises on `UPDATE` and `DELETE` per row and on `TRUNCATE` per statement. The app role cannot disable it: it owns nothing, so `alter table` is refused too.

FROM packages/db/src/index.ts

```

/**
 * Creates (or resets) the login role the application connects as, with data access only: no schema
 * changes, no migration history, no superuser. Run by the owner, next to migrations. It RESETS the
 * role every time: an attribute or membership picked up some other way does not survive a deploy.
 */
export async function provisionAppRole({ ownerUrl, schema = "public", role, password }: {
    ownerUrl: string;
    schema?: string;
    role: string;
    password: string;
}): Promise<void> {
    if (!IDENTIFIER.test(role)) throw new Error(`invalid role name: ${JSON.stringify(role)}`);
    if (!IDENTIFIER.test(schema)) throw new Error(`invalid schema name: ${JSON.stringify(schema)}`);
    if (!/^[^x20-x7e]+$/.test(password)) throw new Error("the app role password must be printable ASCII");
    const owner = new Client({ connectionString: ownerUrl });
    await owner.connect();
    try {
        const r = owner.escapeIdentifier(role);
        const s = owner.escapeIdentifier(schema);
        const exists = await owner.query("select 1 from pg_roles where rolname = $1", [role]);
        await owner.query(
            `${exists.rowCount === 0 ? "create" : "alter"} role ${r} login nosuperuser nocreatedb nocreaterole nobypassrls noreplication inherit
            +
            'connection limit -1 valid until 'infinity' password ${owner.escapeLiteral(scramVerifier(password))}`,
        );
        const memberships = await owner.query<{ parent: string }>(
            "select p.rolname as parent from pg_auth_members m join pg_roles p on p.oid = m.roleid join pg_roles c on c.oid = m.member where c.rolname = $1",
            [role],
        );
        for (const { parent } of memberships.rows) await owner.query(`revoke ${owner.escapeIdentifier(parent)} from ${r}`);
    }
}

```

```

await owner.query(`grant usage on schema ${s} to ${r}`);
await owner.query(`grant select, insert, update, delete on all tables in schema ${s} to ${r}`);
await owner.query(`grant usage, select on all sequences in schema ${s} to ${r}`);
await owner.query(`alter default privileges in schema ${s} grant select, insert, update, delete on tables to ${r}`);
await owner.query(`alter default privileges in schema ${s} grant usage, select on sequences to ${r}`);
await owner.query(`revoke all on ${s}.schema_migrations from ${r}`);
// F26: the app may add to the audit trail and read it, never change or remove it (the table's trigger refuses
// everyone else). Revoked AFTER the grant above, on every provisioning: a deploy restores it if anyone widened it.
await owner.query(`revoke update, delete, truncate on ${s}.audit_log from ${r}`);

```

FROM packages/db/src/audit.int.test.ts

```

test("as the app role, every UPDATE, DELETE and TRUNCATE of audit rows is refused by Postgres (42501), and the rows are unchanged", async () => {
  const { org, rows } = await audited("immutable-app");
  for (const sql of [
    "update audit_log set action = 'signed_in' where org_id = $1",
    "update audit_log set actor_email = 'someone-else@example.com'",
    "delete from audit_log where org_id = $1",
    "delete from audit_log",
  ]) {
    await expect(app.raw(sql, sql.includes("$1") ? [org.id] : [], sql).rejects.toMatchObject({ code: "42501" })); // insufficient_privilege
  }
  await expect(app.raw("truncate audit_log")).rejects.toMatchObject({ code: "42501" });
  // Nor by the back doors a table owner would have: the app role owns nothing.
  await expect(app.raw("alter table audit_log disable trigger audit_log_no_change")).rejects.toMatchObject({ code: "42501" });
  await expect(app.raw("drop table audit_log")).rejects.toMatchObject({ code: "42501" });
  expect(await trail(org.id)).toEqual(rows);
});

```

Durable: no foreign keys, on purpose. The E1.2 review's finding about cascades applies with full force here. `on delete cascade from orgs` would erase an org's whole trail together with the org, which is exactly when someone might want to read it. `on delete set null from users` is an `UPDATE` of the audit row, which the trigger refuses, so the user delete itself would fail. So the table has no foreign keys at all, the email is copied in, and a row may name a user, a document or an org that no longer exists. The test deletes the org and both users and asserts the trail is byte-for-byte what it was.

FROM packages/db/src/audit.int.test.ts

```

test("deleting the org, or the user who acted, erases nothing: no cascade, no set null (the E1.2 review)", async () => {
  const { owner, editor, org, rows } = await audited("immutable-cascade");
  await t.rawQuery("delete from orgs where id = $1", [org.id]);
  await t.rawQuery("delete from users where id = any($1)", [[owner.id, editor.id]]);
  expect(await trail(org.id)).toEqual(rows); // the actor's id and email as they were
});

test("a failed action leaves no audit row: the row and the action commit or roll back together", async () => {
  const { org, owner } = await audited("immutable-atomic");
  // The last owner cannot step down: refused inside the transaction, so nothing is recorded.
  expect(await app.db.forOrg(org.id).setMember({ email: owner.email, role: "viewer", by: owner.id })).toBe("last_owner");
  const ws = await app.db.forOrg(org.id).createWorkspace({ name: "ws" });
  const doc = await app.db.forOrg(org.id).createDocument({ workspaceId: ws.id, title: "d" });
  if (!doc) throw new Error("no document");
  expect(await app.db.forOrg(org.id).createRun({ documentId: doc.id, instruction: "first", createdBy: owner.id })).toMatchObject({ status: "queued" });
  expect(await app.db.forOrg(org.id).createRun({ documentId: doc.id, instruction: "second", createdBy: owner.id })).toBe("busy"); // the statement failed
  expect((await trail(org.id)).map((r) => [r.action, (r.detail as { instruction?: string }).instruction])).toEqual([["role_changed", undefined], ["run_started", "first"]]);
});

```

Reading it. `GET /orgs/:orgId/audit` is owners only, like usage: what happened in an org is the business of whoever runs it. It pages newest first over the same keyset cursor every list in the repo uses (`page()` gained a `newestFirst` flag), and there is no route that changes an entry: the api test tries `POST`, `PUT`, `PATCH` and `DELETE` on the collection and on an entry and asks 404 of all of them. The web view renders every string as a React text node, and its words come from a `Record` over the whole `AuditAction` enum: add an action to the contract and `audit.ts` stops compiling until someone has written its sentence. Drill 1 makes you do exactly that.

FROM apps/api/src/app.ts

```

// F26: the org's audit trail (sign-ins, role and share changes, AI runs, ships, rejected pushes), newest first. Owners
// only, as usage is. Read-only by construction: no route changes or removes an entry, and the database refuses the
// app's role both anyway. Each entry was written by the action itself, in the same statement or transaction.
org.get("/audit", need("owner"), async (c) => {
  const page = await c.var.scope.audit(pageQuery(c));

```

```

    return page ? c.json(page) : badCursor(c);
  });

```

FROM packages/contracts/src/index.ts

```

// --- Audit (F26) -----
export const AuditAction = z.enum(["signed_in", "role_changed", "share_granted", "share_revoked", "run_started", "ship_started", "push_rejected"]);
export type AuditAction = z.infer<typeof AuditAction>;

/**
 * One row of an org's audit trail: who (a person, as their email was then; a push to git; the system), what (the
 * action, the document, and a few flat facts such as the new role or the run's instruction) and when. `detail` holds
 * what people typed (an instruction, a file name from a push): the view renders it as TEXT, never as markup.
 */
export const AuditEntry = z.strictObject({
  id: Id,
  orgId: Id,
  actor: z.strictObject({ kind: z.enum(["user", "git", "system"]), id: Id.nullable(), email: z.string().max(320).nullable() }),
  action: AuditAction,
  documentId: Id.nullable(),
  detail: z.record(z.string().max(40), z.string().max(4000)),
  at: Timestamp,
});
export type AuditEntry = z.infer<typeof AuditEntry>;

/** GET /orgs/:orgId/audit: one page, newest first. Owners only; there is no route that changes or removes an entry. */
export const AuditPage = z.strictObject({ items: z.array(AuditEntry), nextCursor: z.string().nullable() });

```

FROM apps/web/src/audit.ts

```

import { ConflictReason, type AuditAction, type AuditEntry } from "@noon/contracts";
import { refusalSentence } from "../conflict.ts";

/**
 * What an audit entry says, as plain strings: the caller renders them as TEXT (an instruction, an email and a pushed
 * file name are what people typed). A Record over the WHOLE contract: an action added there stops this file compiling
 * until someone has written its sentence. Details are read defensively: an entry written before a detail existed
 * still reads as a sentence.
 */
const SENTENCES: Record<AuditAction, (d: Partial<Record<string, string>>) => string> = {
  signed_in: (d) => "Signed in.",
  role_changed: (d) => (d["previous"] === "none" ? `Added ${d["email"]} ?? "someone" as ${d["role"]} ?? "a member"`. : `Changed the role
of ${d["email"]} ?? "someone" from ${d["previous"]} ?? "?" to ${d["role"]} ?? "?"`.),
  share_granted: (d) => `Shared a document with ${d["email"]} ?? "someone" as ${d["role"]} ?? "?"`. ,
  share_revoked: (d) => `Revoked the share of ${d["email"]} ?? "someone"`. ,
  run_started: (d) => `Started an AI run: "${d["instruction"]} ?? ""`,
  ship_started: (d) => "Started a ship.",
  push_rejected: (d) => {
    const reason = ConflictReason.safeParse(d["reason"]);
    return `A push to ${d["file"]} ?? "the page" was not applied. ${reason.success ? refusalSentence(reason.data) : "It broke the page's
rules."}`;
  },
};

export function auditWords(entry: AuditEntry): { who: string; what: string } {
  const who =
    entry.actor.kind === "user" ? (entry.actor.email ?? "A deleted user")
    : entry.actor.kind === "git" ? `Git, commit ${entry.detail["commit"]} ?? ""`.slice(0, 12)}
    : "The system";
  return { who, what: SENTENCES[entry.action](entry.detail) };
}

```

FROM apps/api/src/audit.int.test.ts

```

const entries = await readAll(org.id); // newest first: reversed, they are the order things happened in
const byUser = { kind: "user", id: owner.id, email: who.owner };
expect([...entries].reverse().map(({ actor, action, documentId, detail }) => ({ actor, action, documentId, detail }))).toEqual([
  { actor: byUser, action: "signed_in", documentId: null, detail: {} },
  { actor: byUser, action: "role_changed", documentId: null, detail: { email: who.editor, role: "editor", previous: "none" } },
  { actor: byUser, action: "role_changed", documentId: null, detail: { email: who.editor, role: "viewer", previous: "editor" } },
  { actor: byUser, action: "share_granted", documentId: doc.id, detail: { email: who.outsider, role: "editor" } },
  { actor: byUser, action: "share_revoked", documentId: doc.id, detail: { email: who.outsider } },
  { actor: byUser, action: "run_started", documentId: doc.id, detail: { run: run.id, instruction } },
  { actor: byUser, action: "ship_started", documentId: doc.id, detail: { ship: ship.id } },
  { actor: { kind: "git", id: null, email: null }, action: "push_rejected", documentId: doc.id, detail: { commit, file:
'src/pages/noon-${doc.id}.tsx', reason: "spread" } },
]);

```


FROM [e2e/audit-view.spec.ts](#)

```
// The instruction's markup stayed text: no image was made of it, and its script never ran.
await expect(table.locator("img")).toHaveCount(0);
expect(await page.title()).not.toBe("owned");
```

TYPESCRIPT (THIS REPO)

JAVA

PYTHON

```
with j as (insert ... returning *) , a as
(insert into audit_log ... from j) select
* from j
```

Envers writes `_AUD` rows from entity listeners in the same transaction; the app role can still `UPDATE` them

django-simple-history writes from signals in the same transaction; same caveat

```
revoke update, delete, truncate on
audit_log from noon_app , after the grant, every
deploy
```

the same SQL, in Flyway or by hand; rarely done

the same SQL; Django's migrations can run it

```
create trigger ... before update or delete
... execute function that raises
```

the same DDL

the same DDL

```
check ((actor_kind = 'user') = (actor_id
is not null)) : a boolean equality as an
invariant
```

the same DDL; Bean Validation cannot express it

the same DDL; a `CheckConstraint` in `Meta`

```
jsonb_build_object('email', u.email,
'role', s.role) and
z.record(z.string().max(40),
z.string().max(4000))
```

a `Map<String, String>` column with a JSON converter

`JSONField`; pydantic `dict[str, str]`

```
Record<AuditAction, (d) => string> :
exhaustive by construction
```

a `switch` over an enum with `-Werror` on a missing case; or `EnumMap` checked at startup

a dict, checked by nothing unless mypy sees a `Literal`

```
rejects.toMatchObject({ code: "42501" }):
the SQLSTATE for insufficient privilege
```

`SQLException.getSQLState()`

`psycopg.errors.InsufficientPrivilege`

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Writing the audit row after the action returned was rejected.

The layered shape, `await db.setMember(...); await audit.write(...)`, is what an `AuditService` looks like in most codebases. A crash, a timeout or a deploy between the two loses the row, and a rolled-back action can leave one, because the second write has no idea the first did not commit. The CTE and the shared transaction make both impossible, and the test for it ("a failed action leaves no audit row") refuses two actions and counts.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Foreign keys on audit_log were rejected: the E1.2 finding, again.

Referential integrity is the reflex, and it is wrong for a trail: a cascade erases history when the thing it is about is deleted, and `set null` is an `UPDATE` the table refuses. The test deletes an org and its users and asserts the rows are unchanged, including the actor's id and email as they were.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Per-row checks in the application were rejected: the app role could still UPDATE.

A repository method that only ever inserts is a promise the code makes about itself; the app role's privileges are a fact about what any code running as it *can* do, including a bug, a migration script or a stolen connection string. Revoking the privilege makes the promise checkable: `has_table_privilege` answers it, and the test asks.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

One global sign-in row was rejected: one per org.

A sign-in belongs to a person, not to an org, so one row seemed natural. But the trail is read per org, by its owners, and an org must see its members' sign-ins without seeing that a member also belongs elsewhere. The statement joins `memberships` and writes one row per org; a user in no org signs in unrecorded, which the test spells out.

The principle: an audit trail is a claim about the past that the present must not be able to edit. Making it in the same statement as the action gives completeness; taking the privilege away from the writer gives immutability against the application; the trigger gives it against everyone else who connects; copying values in and refusing foreign keys gives durability. Envers and simple-history give you the first for free and none of the others unless you do the database's part yourself.

6 · Trade-offs and comparisons: where this design sits

The questions an interviewer asks about an auth design, with the honest answers for this one.

QUESTION	THIS REPO	JAVA WORLD	PYTHON WORLD
Password hashing	<code>scrypt</code> , $N=2^{15}$, $r=8$ (32 MiB), parameters in the string, from <code>node:crypto</code>	Spring's <code>DelegatingPasswordEncoder</code> , <code>bcrypt</code> by default, <code>argon2</code> available	Django: PBKDF2-SHA256 by default (fast to verify, not memory-hard), <code>argon2</code> with a dependency
The browser's session	a row, an opaque 32-byte token kept as SHA-256, a 7-day fixed expiry, <code>HttpOnly</code> + <code>Secure</code> + <code>SameSite=Strict</code>	<code>HttpSession</code> in Spring Session JDBC: the same design; <code>JSESSIONID</code>	<code>django_session</code> : the same design; the key is stored in clear
CSRF	<code>SameSite=Strict</code> , plus JSON-only bodies from epic 1; no token	a synchroniser token (<code>CsrfFilter</code>) by default	a <code>csrftoken</code> cookie echoed in a header
Service-to-service	the worker mints 60-second HS256 tokens with a shared secret and an actor kind	a client-credentials grant, or a shared secret	the same
Roles	three nested roles per org, from the membership row, <code>need(role)</code> per route, a test over the route table	<code>@PreAuthorize</code> + <code>RoleHierarchy</code> ; no table of routes unless you write one	<code>permission_required</code> ; django-guardian for objects
Per-object access	a shares table joined as a subquery: the higher of two roles	Spring ACL (an ACL table per object)	django-guardian
Live revocation	read before the upgrade; re-read on a pub/sub hint; a 30 s sweep; checked at the op's turn	<code>SessionRegistry.expireNow()</code> for HTTP only; WebSockets by hand	Channels groups by hand
Audit	same-statement CTE, revoked grants, a trigger, no foreign keys	Envers: history, editable by the app role	simple-history, auditlog: the same
Tenancy	the membership join in every query (epic 1), shares as a second door	Hibernate filters or <code>@TenantId</code> ; RLS	django-tenants (schemas); RLS

What is deliberately simple. Sessions are checked with one indexed read and no cache: a cache would make revocation a two-place problem, and one read is cheap. Roles are three, nested, per org: a role hierarchy is a total order, which is why `includes` is one comparison; the moment two roles are incomparable ("billing" and "editor"), the design becomes permissions, not roles, and the route table becomes a matrix. Authentication is a password: no OAuth, no email, no second factor, because the spec calls them non-goals and each is a large surface of its own.

What is not solved. There is no email flow, so sign-up must say when an email is taken, there is no password reset, and a member is added by an email that must already have signed up. The rate limiter behind `allowAttempt` is E9.6's and allows everything today, so an online guesser pays only `scrypt`'s price per try. The api announces a change once, best effort, and the sweep covers the gap at 30 seconds. The sync server reads the role twice per connect, at the upgrade and at the join. There is no member list, no removal from an org (a demotion to viewer is the nearest thing), and no sharing screen: shares are made through the api. A rejected push is attributed to `git` and its commit, not to the engineer who pushed. Expired sessions are swept at their owner's next sign-in only. Every one of these is named where it lives, with its ceiling and its upgrade.

The interview answer in one breath: script with self-describing parameters and a dummy hash so sign-in costs the same for everyone; an opaque session token kept as its SHA-256 in a row, in an HttpOnly, Secure, SameSite=Strict cookie, revoked by a DELETE; a 60-second HS256 token between the api and the sync node, with the role deliberately not in it; the role read from the row that proves membership, declared per route and checked by a test over the router; the room reading the role before the upgrade, at the join and on every announcement, and checking it at each op's turn; and an audit trail written by the action's own statement in a table the app role cannot change and a trigger defends against everyone else.

7 · Testing identity without a browser, and the database's refusals

The split from the earlier lessons holds: every decision has a pure test that ran on every commit, every I/O rule has an integration or end-to-end test that is written and unrun. Five shapes are new to this epic.

A structural fake of the database. The auth routes are tested against an object with five methods, cast `as unknown as Db`: TypeScript's structural typing means the fake needs only what the routes call, and the cast records that this is a fake, not a `Db`. It also records what it was asked (`calls`), so "a 400 body is refused before anything is hashed or stored" is an assertion on an empty array.

FROM `apps/api/src/auth.test.ts`

```
function memoryDb() {
  const users = new Map<string, { user: User; passwordHash: string }>();
  const sessions = new Map<string, { userId: string; expires: number }>();
  const calls: string[] = [];
  const db = {
    signUp: ({ email, name, passwordHash }: { email: string; name: string; passwordHash: string }) => {
      calls.push("signUp");
      const key = email.toLowerCase();
      if (users.has(key)) return Promise.resolve("taken" as const);
      const user = { id: randomUUID(), email: key, name };
      users.set(key, { user, passwordHash });
      return Promise.resolve(user);
    },
    credentialsFor: (email: string) => { calls.push("credentialsFor"); return Promise.resolve(users.get(email.toLowerCase())); },
    startSession: ({ userId, tokenHash, ttlSeconds }: { userId: string; tokenHash: Buffer; ttlSeconds: number }) => {
      sessions.set(tokenHash.toString("hex"), { userId, expires: Date.now() + ttlSeconds * 1000 });
      return Promise.resolve();
    },
    userForSession: (tokenHash: Buffer) => {
      const s = sessions.get(tokenHash.toString("hex"));
      return Promise.resolve(s && s.expires > Date.now() ? [...users.values()].find((u) => u.user.id === s.userId)?.user : undefined);
    },
    endSession: (tokenHash: Buffer) => { sessions.delete(tokenHash.toString("hex")); return Promise.resolve(); },
    listOrgsFor: () => Promise.resolve({ items: [], nextCursor: null }),
    take: () => Promise.resolve({ ok: true }), // the limits themselves: http-limits.test.ts
  };
  return { db: db as unknown as Db, sessions, calls };
}
```

A timing test with real medians. The enumeration defence cannot be tested with a mock of `verifyPassword`: the claim is about wall-clock time, so the test signs up, times five unknown emails and five wrong passwords through the real script, and compares medians (medians, because one garbage collection would ruin a mean). Without the dummy hash the unknown email answers in well under a millisecond; the assertion is loose (`0.3`) so a loaded host does not make it flaky, and tight enough that the bug fails it by an order of magnitude.

The router read as data. Hono exposes `app.routes`, a list of `{ method, path, handler }`; `need` registered each guard in a `WeakMap`; the test joins the two and compares the whole table. It is a test of the *configuration*, not of any request, and it is the only kind of test that catches a route nobody wrote a test for.

The real entry point in a real process. A test that hands `startServer` an identity strategy cannot prove that `main.ts` chooses the right one from `NODE_ENV`. `useApiProcess` spawns `main.ts` with a clean environment (nothing from the runner leaks in, in particular vitest's own `NODE_ENV=test`), waits for `/health`, and the test asks 401 of the header with `NODE_ENV` unset, `production` and `test`, and 200 with `development`.

FROM `apps/api/src/testing.ts`

```
/**
 * The REAL entry point (main.ts) in a REAL process, for one test file: `boot(env)` starts it in a clean environment
 * (nothing from the test runner, such as NODE_ENV=test, leaks in) plus `env`, and answers once /health does. The
 * process is killed after each test.
 */
export function useApiProcess(): (env: Record<string, string>) => Promise<{ url: string; stderr: () => string }> {
  const main = new URL("./main.ts", import.meta.url).pathname;
  let child: ChildProcess | undefined;
  afterEach(() => child?.kill("SIGKILL"));
  return async (env) => {
    const port = String(20000 + Math.floor(Math.random() * 20000));
```

```

    let stderr = "";
    child = spawn(process.execPath, [main], { env: { PATH: process.env["PATH"] ?? "", DATABASE_URL: TEST_DATABASE_URL, PORT: port,
    SESSION_TOKEN_SECRET: "m".repeat(32), SYNC_PUBLIC_URL: "ws://localhost:3001", REDIS_URL: "redis://localhost:6380", ...env } });
    child.stderr?.on("data", (chunk: Buffer) => (stderr += chunk.toString()));
    const url = `http://127.0.0.1:${port}`;
    for (let i = 0; i < 100; i++) {
      try {
        if ((await fetch(`${url}/health`)).ok) return { url, stderr: () => stderr };
      } catch {
        await new Promise((r) => setTimeout(r, 50));
      }
    }
    throw new Error(`api did not start: ${stderr}`);
  };
}

```

A hook inside the role read. The revoke race needs a role read that has its answer but has not yet reached the server, exactly as a slow Postgres would produce. The test's `roles` function wraps the real one and awaits an `afterRead` hook the test sets per case; the hook resolves a promise the test awaits ("the read has answered"), then waits on one the test resolves later ("now deliver it"). Two `Promise.withResolvers()` and no fake clock.

FROM `apps/sync/src/role-change.int.test.ts`

```

/** E8.3's race tests: runs after each role read has its answer and before the sync server gets it (a slow Postgres). */
let afterRead: (userId: string) => Promise<void> | undefined;
let t: TestDb, api: RunningServer, sync: RunningSyncServer;

```

FROM `apps/sync/src/role-change.int.test.ts`

```

sync = await startSyncServer({ port: 0, secrets: [TEST_SECRET], store: t.db.documentStore(), roles: async (orgId, documentId, userId) => {
  const role = await t.db.roleIn(orgId, documentId, userId);
  await afterRead?.(userId);
  return role;
}, recoverMs: 50 });

```

Asking Postgres what a role may do. `has_table_privilege(role, table, 'update')` is the database's own answer to "can this role change the trail", and the app-role tests ask it before and after re-provisioning, then try the statements and match on SQLSTATE `42501`. A throwaway app role is provisioned per test file with the same function the migrate container runs, over the test's own schema.

FROM `packages/db/src/audit.int.test.ts`

```

test("the app role holds SELECT and INSERT on audit_log, and neither UPDATE, DELETE nor TRUNCATE", async () => {
  const privileges = await t.rawQuery(
    "select has_table_privilege($1, $2, 'select') as s, has_table_privilege($1, $2, 'insert') as i, has_table_privilege($1, $2, 'update') as u, " +
    "has_table_privilege($1, $2, 'delete') as d, has_table_privilege($1, $2, 'truncate') as t",
    [app.role, `${t.schema}.audit_log`],
  );
  expect(privileges).toMatchObject({ rows: [{ s: true, i: true, u: false, d: false, t: false }] });
});

```

FROM `packages/db/src/testing.ts`

```

/** Provisions a throwaway app login role over this test schema, exactly as migrate-cli does in production. */
export async function createAppRole(t: TestDb): Promise<AppRole> {
  const role = `noon_app_test_${randomBytes(4).toString("hex")}`;
  const password = "test-only-password";
  const reprovision = () => provisionAppRole({ ownerUrl: TEST_DATABASE_URL, schema: t.schema, role, password });
  await reprovision();
  const u = new URL(TEST_DATABASE_URL);
  u.username = role;
  u.password = password;
  const url = u.toString();
  const db = createDb({ connectionString: url, schema: t.schema });
  return {
    role,
    password,
    db,
    url,
    reprovision,
    async raw(sql, params = []) {
      const c = new Client({ connectionString: url, options: { '-c search_path=${t.schema}' } });
      await c.connect();
      try {
        return await c.query(sql, params);
      } finally {
        await c.end();
      }
    }
  };
}

```

```
}  
},
```

FROM `apps/api/src/identity.test.ts`

```
test("a session lookup is a pure read, and a malformed cookie never reaches the database", async () => {  
  expect((await who("production", cookie(signedIn.token))).asked).toEqual(["session"]);  
  for (const bad of [ "", "short", `${signedIn.token}x`, "a"; drop table users; -- ]) expect((await who("production", cookie(bad))).asked,  
  bad).toEqual([]);  
});
```

TYPESCRIPT (THIS REPO)

JAVA

PYTHON

`{ ... } as unknown as Db`: a structural fake with only the methods used

Mockito, or an interface with one implementation per test

`MagicMock`, or a `Protocol` and a small class

`performance.now()` around a request, medians over five runs

`System.nanoTime()`; JMH for anything serious

`time.perf_counter()`

`spawn(process.execPath, [main], { env: { PATH, ... } })`: a child with a clean environment

`ProcessBuilder` with `environment().clear()`

`subprocess.Popen(..., env={...})`

`test.each(["unset", {}], ...)( "with NODE_ENV %s ...")`

`@ParameterizedTest` with `@MethodSource`

`@pytest.mark.parametrize`

`await expect(promise).rejects.toMatchObject({ code: "42501" })`

`assertThrows` and `getSQLState()`

`pytest.raises(InsufficientPrivilege)`

`Promise.withResolvers<undefined>()` twice: "answered" and "deliver"

two `CountDownLatch`es

two `asyncio.Event`s

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The dev-header test had to spawn the real entry point.

The first version passed `chooseIdentity("production")` to `startServer` and asserted 401. It proved the strategy, not the wiring: `main.ts` could have ignored `NODE_ENV` entirely and the test would stay green. Spawning `main.ts` with a clean environment is slower (a process per case) and is the only test that would go red if someone "simplified" the boot.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A mocked timing test would prove nothing.

Replacing `scrypt` with a stub that sleeps would test the stub. The unit test pays for ten real derivations (a second or so) because the claim is about the real hash's cost; the drill's version stubs it, because there the claim is about the *shape* of the code, and the comment says so.

Honest status, repeated here because it matters: the unit tests (`password.test`, `auth.test` against the in-memory db, `identity.test`, the route table in `app.test`, the viewer's refusal in `room.test`, the contracts, `audit.test` in the web app, `config.test`) ran green on every commit. The integration suites (`auth.int` including the spawned entry point, `service-credential.int`, `rbac.int`, `role-change.int` including the token race and the sweep, the api's `audit.int`, the db package's `audit.int` and `app-role.int`) and the four e2e specs (`sign-in`, `viewer-rejected`, `share-and-revoke`, `audit-view`) are written, reviewed and committed, and have not been run. Section 8 is what running them looks like.

8 · By hand: sign up, demote, revoke, and try to edit the trail

The exact commands and what each should show. The api is on host port 3000, the sync node on 3001, the web app on 5173; the Postgres owner role is `noon` and the app's role `noon_app`. Sign in and out from the web app happens on the home page, with no `?user=` in the address; the dev header still works in this development stack, which is what lets one terminal act as several people.

0 · BRING THE STACK UP AND SIGN UP AS YOURSELF, WITH A COOKIE JAR

```
./init.sh # migrate runs 0015 (credentials, auth_sessions), 0016 (document_shares), 0017 (audit_log)  
open http://localhost:5173/ # the sign-in form; Sign up, then New document; note ?doc=<id>  
curl -si -c jar -H 'content-type: application/json' -d '{"email":"cli@example.com","name":"Cli","password":"correct horse battery"}'  
localhost:3000/auth/signup  
# 201; Set-Cookie: noon_session=<43 chars>; Max-Age=604800; Path=/; HttpOnly; SameSite=Strict
```

```
(no Secure: development)
curl -s -b jar localhost:3000/auth/me      # {"user":{"email":"cli@example.com"}}
curl -s localhost:3000/auth/me            # {"user":null}, and 200: nobody is not an error
curl -s -H 'x-dev-user: cli@example.com' localhost:3000/orgs # in development the header still works; try NODE_ENV=production and it is
401
```

1 • WHAT POSTGRES KEEPS, AND WHAT AN UNKNOWN EMAIL COSTS (F23)

```
docker compose exec -T postgres psql -U noon -d noon -c "select password_hash from credentials" # script$32768$8$1$<salt>$<key>
docker compose exec -T postgres psql -U noon -d noon -c "select encode(token_hash, 'hex'), expires_at from auth_sessions" # a SHA-256,
not the cookie
time curl -s -H 'content-type: application/json' -d '{"email":"nobody@example.com","password":"x"}' localhost:3000/auth/signin
time curl -s -H 'content-type: application/json' -d '{"email":"cli@example.com","password":"wrong"}' localhost:3000/auth/signin
# both {"error":"invalid_credentials"}, both about the same time: one script each
curl -si -b jar -X POST localhost:3000/auth/signout # 204; Set-Cookie: noon_session=; Max-Age=0
curl -s -b jar localhost:3000/orgs # {"error":"unauthenticated"}: the row is gone, the copy in the jar is dead
```

2 • ROLES, LIVE (F24)

```
curl -s -c jar -H 'content-type: application/json' -d '{"email":"cli@example.com","password":"correct horse battery"}'
localhost:3000/auth/signin >/dev/null
ORG=$(curl -s -b jar localhost:3000/orgs | jq -r '.items[0].id') # the org New document made for you (or create one: POST /orgs)
curl -s -H 'x-dev-user: viewer@example.com' localhost:3000/auth/me >/dev/null # the header creates them (development only)
curl -s -b jar -X PUT -H 'content-type: application/json' -d '{"email":"viewer@example.com","role":"viewer"}'
localhost:3000/orgs/$ORG/members
open 'http://localhost:5173/?user=viewer@example.com&doc=<id>' # live; your edits appear; Add Card there: "You can view this document
but not edit it."
docker compose exec -T redis redis-cli --no-auth-warning SUBSCRIBE access # in another terminal: the hint,
{"orgId":...,"userId":...}, on each change
curl -s -b jar -X PUT -H 'content-type: application/json' -d '{"email":"viewer@example.com","role":"editor"}'
localhost:3000/orgs/$ORG/members
# the SUBSCRIBE prints one message; within 10 s, without a reload, the viewer's Add Card
lands
curl -s -b jar -X PUT -H 'content-type: application/json' -d '{"email":"cli@example.com","role":"viewer"}'
localhost:3000/orgs/$ORG/members # {"error":"last_owner"}, 409
```

3 • SHARE, MINT A TOKEN, REVOKE, TRY EVERY WAY BACK (F25)

```
curl -s -H 'x-dev-user: guest@example.com' localhost:3000/auth/me >/dev/null
curl -s -b jar -X PUT -H 'content-type: application/json' -d '{"email":"guest@example.com","role":"editor"}'
localhost:3000/documents/<id>/shares # {"userId":"<guest>"},...
open 'http://localhost:5173/?user=guest@example.com&doc=<id>' # live, editing; GET /orgs/$ORG as the guest is still 404: a share is
not membership
TOKEN=$(curl -s -X POST -H 'x-dev-user: guest@example.com' localhost:3000/documents/<id>/session | jq -r .token) # minted BEFORE the
revoke; good for 60 s
curl -si -b jar -X DELETE localhost:3000/documents/<id>/shares/<guest> # 204; the SUBSCRIBE prints the hint
# within 10 s the guest's tab shows "This document cannot be opened" and its status line is
gone: 4404, no reconnect
curl -si -X POST -H 'x-dev-user: guest@example.com' localhost:3000/documents/<id>/session # 404
curl -si -H 'connection: upgrade' -H 'upgrade: websocket' -H 'sec-websocket-version: 13' -H 'sec-websocket-key: dGh1IHNBbXBzS2Bub25jZQ=='
\
-H "sec-websocket-protocol: noon.v1, $TOKEN" localhost:3001/documents/<id> # HTTP/1.1 401: read before the upgrade, whatever the
signature says
```

4 • THE TRAIL, AND TRYING TO CHANGE IT (F26)

```
curl -s -b jar localhost:3000/orgs/$ORG/audit | jq '.items[] | [.at, .actor.email, .action, .detail]' # newest first: share_revoked,
share_granted, role_changed x2, signed_in...
open 'http://localhost:5173/?audit=$ORG' # or Home, then "Audit trail of ..."; the instruction of a run shows as text, markup and all
curl -si -b jar -X DELETE localhost:3000/orgs/$ORG/audit # 404: no such route
docker compose exec -T postgres psql -U noon -d noon -c "update audit_log set action = 'signed_in'"
# ERROR: permission denied for table audit_log (42501)
docker compose exec -T postgres psql -U noon -d noon -c "delete from audit_log" # the OWNER: ERROR: audit_log is append-only:
DELETE refused
docker compose exec -T postgres psql -U noon -d noon -c "select has_table_privilege('noon_app', 'audit_log', 'update')" # f
```

5 • THEN THE CHECKS THAT PROVE IT WITHOUT HANDS

```
pnpm exec vitest run apps/api/src/password.test.ts apps/api/src/auth.test.ts apps/api/src/identity.test.ts apps/api/src/app.test.ts
apps/web/src/audit.test.ts # the pure halves
pnpm exec vitest run --config vitest.integration.config.ts apps/api/src/auth.int.test.ts apps/worker/src/service-credential.int.test.ts
apps/api/src/zbac.int.test.ts \
  apps/sync/src/role-change.int.test.ts apps/api/src/audit.int.test.ts packages/db/src/audit.int.test.ts packages/db/src/app-
role.int.test.ts
pnpm exec playwright test e2e/sign-in.spec.ts e2e/viewer-rejected.spec.ts e2e/share-and-revoke.spec.ts e2e/audit-view.spec.ts
make check # the whole gate, as the pre-commit hook runs it
```

What to do when a step does not match this page: the page was cut from the code at commit `f21d6d2` , and the code is right until a test says otherwise. A step that fails is a finding: write it as *sequence* $\rightarrow$ *false belief* $\rightarrow$ *consequence* $\rightarrow$ *smallest fix* (SPEC §4a's form) and file it against the bead that owns the code.

9 · What the type system could not see, epic 8 edition

Everything below compiled under `strict` , passed the linter and the first tests, or was rejected at design time by asking what the types could not ask.

WHAT WENT WRONG, OR WOULD HAVE	WHY	FIX
a fast hash for passwords	a hash is a hash	scrypt: slow and memory-hard, parameters in the string
an unknown email answered without hashing	two branches look alike	verify against a dummy hash; <code>found && matches</code>
the first miss cost two hashes	lazy initialisation looks free	the dummy warmed in <code>buildApp</code>
a row asking for <code>N = 2^40</code>	a number is a number	a ceiling on the parameters; scrypt's refusal is a no
<code>é</code> as one code point and as two	two equal-looking strings	<code>NFKC</code> before hashing
a sign-in validated like a sign-up	one body schema is simpler	<code>SignInBody</code> keeps only the caps
the session token stored as itself	a string is a string	its SHA-256, as <code>bytea</code>
a cookie a script could read	a cookie is a cookie	<code>HttpOnly</code>
a planted cookie becoming a session	same name, same shape	a new token on every sign-in
a sliding expiry writing on every read	an <code>UPDATE</code> looked harmless	fixed at sign-in; identity stays a pure read
a stateless token for the browser	no table looked simpler	a row you can delete
<code>POST /session</code> with no CSRF defence	no body, so no JSON check	<code>SameSite=Strict</code>
an image promoted with <code>NODE_ENV=test</code> accepting the header	"not production" reads as "safe"	only the literal <code>development</code>
a name with a right-to-left override	a string	<code>DisplayName</code> refuses <code>\p{Cf}</code>
a route without a guard	a handler compiles without one	the route table test over <code>app.routes</code>
404 to a member	"not found" felt safer	403 for members, 404 for strangers
two owners demoting each other	READ COMMITTED shows each an owner	an advisory lock per org
an editor promoting themselves	PUT members was an org route	<code>need("owner")</code>
the role in the token	a claim is a claim	read from Postgres at every join
<code>mayEdit</code> checked when an op arrives	queued ops had passed	checked at the op's turn
a forged pub/sub message granting a role	a message is data	a hint only: the node re-reads
a missed announcement	pub/sub keeps nothing	<code>"all"</code> on ready, and a 30 s sweep
an older role read overwriting a newer	two awaits, either order	<code>asked / applied</code> numbering
a token minted before the revoke opening a socket	a valid signature	the row read before the upgrade: 401
a revoke between the read and the registration	the registry did not have the socket yet	read again after registering
a share that made a member	a flag on the membership row	a table of its own, never owner
the audit row after the action	two awaits look sequential	one statement (a CTE) or one transaction
a trail cascading away with its org	referential integrity	no foreign keys; the email copied in
<code>set null</code> on a user delete	looked harmless	it is an <code>UPDATE</code> ; no foreign key
the app role able to <code>UPDATE</code> the trail	a blanket grant, default privileges	revoked on every provisioning, plus a trigger
a sign-in recorded once, globally	it belongs to a person	one row per org
an instruction rendered as markup	a string	text nodes only; the e2e plants <code>onerror</code>
a new <code>AuditAction</code> without a sentence	a <code>switch</code> with a default	a <code>Record</code> over the enum

the same role set again recorded as a change

an upsert is an upsert

compare `found.role` first

The pattern, extended once more: lesson 7 said types cannot see time or which process is speaking. Epic 8 adds the two that identity turns on: types cannot see *who is asking* (a stranger's request and a member's are the same TypeScript) and cannot see *what a value costs to compute* (a fast hash and a slow one have the same signature; a branch that skips the hash returns the same type). Both were answered as before: put the truth in one place (the membership row, the session row, the audit table), read it at the moment of asking, make the cost the same on every path, and write a test that provokes the exact case: the timing, the race, the forgotten guard, the privilege.

Your turn: three drills

Do these alone. They live in `drills/lesson-8/` and both coded drills start RED; `sh drills/lesson-8/check.sh` shows your progress. Neither needs Docker.

D1 • Extend a type, end to end. The trail records sign-ins and says nothing about sign-outs. Carry a `signed_out` action from the contract to the row that writes it: `AuditAction` gains it (`packages/contracts`); `apps/web/src/audit.ts` stops compiling until you write its sentence, because `SENTENCES` is a `Record` over the whole enum; a migration `0018_*.sql` drops and re-creates the CHECK constraint `audit_log_action_check` with the new value, and touches nothing else on that table; and `endSession` in `packages/db/src/index.ts` deletes the session and writes one row per org the user belongs to *in one statement*, a CTE over the deleted row, as `startSession` does for a sign-in. Then extend `apps/api/src/audit.int.test.ts` to sign out and expect the row. make `check` must stay green.

Predict first: why must the audit row be selected *from* the deleted row rather than inserted after the delete returns, and what does a sign-out with a cookie that expired an hour ago write under each design?

D2 • Fix a planted bug. `drills/lesson-8/d2-sign-in/sign-in.ts` is `POST /auth/signin` cut down to its decision (the store, the hash check, the one refusal) with one bug from this chapter planted in it. Its hashing is injected and timed, so the test can count hashes and measure the answer. Read the failing assertion and say, before you open the code, what a caller with a stopwatch learns from the answer as it is, and what list they build from it. The fix is two lines.

D3 • Explain a decision before reading mine. Answer out loud or on paper, then open the answer.

▼ 1. The browser's session is 32 random bytes in a row, kept as a SHA-256; the password is kept as `scrypt`; the sync token is a signed JWT with no role in it. Defend each choice against the obvious alternative (a JWT for the browser, `scrypt` for the token, a role claim in the sync token), and say what one property decides the first.

Revocation decides the first. A signed stateless token is valid until it expires wherever it is; signing out would delete the browser's copy only, and a stolen copy or another tab keeps working until `exp`. Revoking early needs a server-side list checked on every request, which is a session table with worse ergonomics, so the browser gets the row from the start and sign-out is a `DELETE` that kills the token on its next request. The hash is chosen by entropy: a password can be guessed, so it gets a KDF that is slow and memory-hard (`scrypt`, 32 MiB per guess, its parameters in the string so they can rise); the token is 256 random bits that cannot be guessed at any speed, so one fast SHA-256 is right, and its only job is that a leaked table hands nobody a live session; a KDF there would cost tens of milliseconds on every request for nothing. The sync token stays a JWT because its stateless property is the point (the api signs, the sync node verifies, they never call each other) and its lifetime is 60 seconds, which is the only revocation a signature allows; the role is not in it because a token is checked once at connect and the socket then lives for hours, so a role claim is stale the moment an owner changes anything. The sync server reads the role itself, at the moments section 4 names.

▼ 2. The api reads the role from the same row that proves membership and declares a role per route; the sync server reads the role before the upgrade, at the join, on every announcement and every 30 seconds, and the room checks `mayEdit` at the op's turn. Why each of those, and why did E8.3 have to move the first read to *before* the upgrade when the token only lives 60 seconds?

Authorise where you fetch: the membership query already returns the row, and the role is a column of it, so one read answers both "may you see this" (404 for a stranger, F2) and "as what" (403 for a member without the role). Declaring the role per route with `need` makes the requirement visible, and the test that walks `app.routes` makes a forgotten guard a red test instead of an open route, which is the failure mode of every handler-level scheme. The socket is the hard case because it is authenticated once and lives for hours: the role must be re-read where the socket lives. The join read sets `mayEdit`; the announcement (a Redis hint that names who changed and never the role, so a forged message can only cause a re-read) makes the node re-read the sessions it concerns within F24's ten seconds; the 30-second sweep covers an announcement the api could not publish; and the room checks the flag at the head of its queue so that an op queued before a demotion is refused too. The upgrade read exists because a revoked outsider's client holds a token that is validly signed for up to 60 more seconds; admitting on the signature and closing at the join would open a socket that should never exist and let a reconnecting client

reopen one for a minute. Reading the row while the request is still HTTP makes that token a plain 401, and the socket's lifetime, the token's lifetime and the cookie's lifetime are all beaten by the same fact: the row is read at the moment of asking.

▼ 3. The audit row is inserted in a CTE of the action's own statement, or in its transaction; the table has no foreign keys; provisionAppRole revokes UPDATE, DELETE and TRUNCATE on every deploy; and a trigger refuses them to everyone. Why each, and why are both the revoke and the trigger needed?

Same statement or transaction, for completeness in both directions: an action cannot commit without its row (a crash between two separate writes would lose it), and a row cannot outlive an action that rolled back (a refused last_owner , a run that hit run_in_progress , a share of nobody write nothing). No foreign keys, for durability: on delete cascade from orgs would erase the trail exactly when someone wants to read it, and on delete set null from users is an UPDATE the trigger refuses, so the email is copied in and a row may name things that no longer exist. The revoke, for immutability against the application: the app role gets a blanket grant plus default privileges for new tables, so audit_log would be writable unless the three privileges are taken away after the grant, and doing it on every provisioning means a grant made by hand is gone after the next deploy; has_table_privilege answers whether it worked. The trigger, for immutability against everyone the grants do not bind: the table's owner (grants never restrict an owner) and any role widened between deploys. The app role cannot disable it because it owns nothing, so alter table is refused. Either layer alone leaves a path: grants alone let the owner edit; the trigger alone could be disabled by a role that owned the table. Together, the only way to change the trail is to be a superuser, which nothing in the stack connects as.

Recall first

Look away. Name the three moments the sync server reads a peer's role, and what happens at each when the answer is "no access". Then name the two layers that make audit_log append-only and who each one binds. Then check.

▼ Check yourself

Before the upgrade (admits): no access is an HTTP 401, no socket is ever opened, and a token minted before a revoke is refused here. At the join (applyRole , after the socket is registered): no access is a 4404 close, fatal to the client. On every announcement and every 30 seconds (recheck , numbered so an older read never overwrites a newer one): no access is a 4404 close; a demotion flips mayEdit , which the room checks at each op's turn. The two layers: provisionAppRole revokes UPDATE, DELETE and TRUNCATE from the app's login role after the blanket grant, on every deploy, binding the application; the trigger raises on UPDATE, DELETE and TRUNCATE for every role that connects, the owner included.

Then test it

1. Why script for passwords rather than SHA-256, and why memory-hard rather than merely slow?

Node has no SHA-256

A password can be guessed offline from a leaked table, so the hash must be slow; and a GPU has thousands of slow cores, so each guess must also cost RAM (32 MiB here), or the attacker parallelises the slowness away

SHA-256 is broken

script is faster to verify

The stored string names N , r and p , so the cost can rise without a migration; Django and Spring's DelegatingPasswordEncoder store hashes the same way.

2. Someone signs in with an email that has no account. What does the route do, and why?

Verifies the password against a real scrypt hash of a password nobody knows, warmed at startup, so the miss costs one hash like a wrong password and answers the same bytes: the response time cannot say which emails have accounts

Returns 404 so the client can offer sign-up

Creates the account

Returns 401 at once, without hashing: there is nothing to compare

The unit test times five unknown emails and five wrong passwords through the real scrypt and compares medians; drill 2 plants the early return.

3. Why is the browser's session a row in Postgres and not a signed token like the sync token?

Rows are faster to verify than signatures

Revocation: sign-out is a DELETE and a stolen or copied token is dead on its next request; a signed token cannot be revoked before it expires without a deny list, which is a table anyway

JWTs cannot be stored in cookies

The sync token is also a row

The sync token stays a JWT because the api and the sync node never call each other, and its 60-second life is its revocation; section 4 is about the one case where 60 s was too long.

4. The session token is kept as its SHA-256, the password as scrypt. Why not the same hash for both?

The token is 256 random bits that cannot be guessed at any speed, so a fast hash is enough to keep a leaked table useless; a password can be guessed, so it needs a KDF that is slow and memory-hard. A KDF on the token would cost tens of milliseconds on every request for nothing

SHA-256 is more secure than scrypt

They are the same hash

Postgres cannot store scrypt output

Pick the hash by the entropy of what it protects; `isSessionToken` also refuses anything that is not exactly the 43-character shape before the database is asked.

5. An editor has three ops waiting in the room's queue when an owner demotes them to viewer. What happens to those ops?

All three are refused as forbidden: `mayEdit` is checked when each op's turn comes, at the head of the queue, not when it arrived

They wait until the editor is promoted again

They land: they were accepted when they arrived

The socket is closed with 4404

The first shape checked in `submit` and let every queued op through; the room test and the integration test ("its next op is refused within 10 s") pin the turn-time check.

6. An outsider's share is revoked. Their client holds a sync token minted 20 seconds earlier, valid for 40 more. What does the sync server answer when the client presents it?

It opens the socket for the 40 seconds left

HTTP 401 at the upgrade: admits reads the share row before any socket exists, so a valid signature alone opens nothing, and the token's lifetime never matters

It opens the socket, then closes it with 4409 so the client asks /session again

It opens the socket in read-only mode

F25's clause: neither a reconnect nor a token minted before the revoke gets them back in. `/session` is a 404 too, which peer-client reads as "give up".

7. Why is the audit row inserted in a CTE of the action's own statement rather than by an audit service after the action returns?

So the action cannot commit without its row (a crash between two writes would lose it) and a row cannot outlive an action that rolled back (a refused last_owner or a busy run writes nothing)

So the row gets the same timestamp

Postgres requires it for tables without foreign keys

It saves one round trip

The test "a failed action leaves no audit row" refuses two actions and counts; `setMember` uses a transaction instead, and writes only when the role really changed.

8. Why does `provisionAppRole` revoke UPDATE, DELETE and TRUNCATE on `audit_log` on every deploy, and why is there a trigger as well?

Only the trigger is needed; the revoke is documentation

Only the revoke is needed; the trigger is for performance

The trigger is a backup in case the revoke fails to run

The app role gets a blanket grant and default privileges, so the revoke must come after and be repeated so a hand-made grant dies at the next deploy; grants never bind the table's owner, so the trigger refuses everyone who connects, and the app role cannot disable it because it owns nothing

`has_table_privilege` answers the first; the tests re-grant and re-provision, then try every statement as the app role (42501) and as the owner (append-only exception).

Next: Lesson 9, after epic 9. Job hardening: idempotency keys that survive a race, a worker killed mid-job that resumes without a duplicate, queues rebuilt from Postgres after a Redis wipe, progress that survives a reload, and the rate limits that plug into `allowAttempt`.